World Models in Action

4. Predicting in representation space

This chapter covers

- Spatial and spatiotemporal JEPA prediction, from I-JEPA to V-JEPA
- Cross-modal prediction into language embeddings
- Collapse, predictive sufficiency, and the statistical geometry of embeddings
- LeJEPA/SIGReg and VISReg as distributional regularizers
- Action-conditioned prediction, planning, memory, hierarchy, and predictive-agent design

4.1. From representation learning to prediction

Chapter 3 ended with an encoder. Given an observation x, an encoder produces a feature vector or feature map $z = f_\theta(x)$. The representation is useful only insofar as it preserves distinctions needed by a later task. That leaves a question which the encoder alone does not answer: how should a model learn which information in one part of an observation predicts information somewhere else?

A joint-embedding predictive architecture (JEPA) makes the relation explicit. It selects a context, selects a target, encodes both into a shared feature space, and trains a predictor to estimate the target features from the context features. The prediction is made in representation space rather than directly in pixel space.

This change is easy to state and easy to misunderstand. It does not mean that the model predicts an abstract state whose meaning is already known. The target encoder defines the feature being predicted. The mask defines which information is withheld. The predictor learns the relation between the two. Whether the resulting representation is a useful state is a further question, not a consequence of the name JEPA.

4.1.1. The problem left by an encoder

Suppose a camera observes a table with a cup, a hand and a robot gripper. An encoder can map the image to a grid of features. But the grid by itself does not tell us which features should be related. One training task might require identifying the cup. Another might require locating its rim. A third might ask what visual change is likely after the gripper closes.

These tasks depend on overlapping information, but they do not ask for the same target. An image-level category may ignore the cup's exact position. A localization task cannot. A prediction of the

result of closing the gripper must preserve information about contact and geometry. The representation-learning objective must therefore specify which relation the encoder is being trained to support.

Pixel reconstruction is one possible choice. It asks a decoder to reproduce the withheld pixels. That objective can be useful when appearance itself matters, but it also rewards detail that may not matter to the task. A model may spend capacity reproducing texture, lighting or small image variations even when the useful prediction concerns object identity, spatial arrangement or motion.

Feature prediction changes the target. Instead of asking the decoder to reproduce a missing image region, the model asks a predictor to estimate the target encoder's features at the corresponding positions. The training signal is still derived from the observation, but the object being matched is a learned representation.

Neither target is automatically superior. Reconstructing appearance can preserve detail that a feature encoder discards. Predicting features can avoid spending the entire objective on pixel-level fidelity. The decision is about what the downstream system needs to retain.

4.1.2. Context, target and predictor

Let x be an observation, such as an image or a video clip. A sampling rule divides its information into a context and a target. Write x_C for the input available to the context branch and T for the positions whose features are to be predicted.

The context encoder produces

$$z_C = f_\theta(x_C).$$

A target encoder produces a feature map from the target-side observation. The target features at the selected positions are

$$z_T = \left[f_{\bar{\theta}}(x)\right]_T.$$

The notation $[\cdot_\mathrm{T}]$ means that features are selected at target positions T. It does not mean that the target encoder necessarily receives only a cropped target patch. In I-JEPA and V-JEPA, the target encoder processes the full image or clip before the selected target features are read out.

The predictor receives the context representation and information identifying the requested target positions:

$$\hat{z}_T = g_\phi(z_C, q_T).$$

Here q_T is a query containing the target locations. For an image, these locations are spatial. For a video, they include temporal as well as spatial coordinates. In a controlled dynamics model, the condition must also include the action whose consequence is being predicted.

The learning objective compares the predicted target features with the target encoder's features:

$$\mathcal{L}_{\text{pred}} = \mathbb{E}_{(x,C,T)\sim\mathcal{D}} \left[\ell\big(g_\phi(f_\theta(x_C), q_T), [f_{\bar{\theta}}(x)]_T\big) \right].$$

The expectation is over the training distribution $\mathcal{D}$, including the observations and the sampled context-target configurations. The function ℓ measures disagreement in the chosen feature space. Squared error, absolute error and smooth-L_1 loss make different statistical choices; the architecture does not fix one universal discrepancy.

A JEPA is therefore not specified by three network boxes alone. It is specified by the input distribution, the context sampler, the target sampler, the target encoder, the query representation, the predictor and the feature loss. Changing any of these can change what the model is required to learn. This formulation follows the predictive-architecture proposal and the original image and video JEPA constructions (LeCun, 2022; Assran et al., 2023; Bardes et al., 2024).

4.1.3. Why the target is a feature

Let $r(x)$ be a representation that an ideal downstream task would use. A pixel loss compares the prediction with the observation x. A feature loss compares it with $f_{\bar{\theta}}(x)$. This gives the target encoder an important role: it decides which differences in the observation remain visible to the prediction objective.

If a change in illumination is removed by the target encoder, the predictor is not penalized for failing to reproduce it. If object position is retained, the predictor must estimate position-dependent features. The target encoder therefore supplies an implicit definition of which distinctions count.

That definition can be helpful or harmful. An encoder that removes background texture may make prediction easier without damaging a control task. An encoder that removes the relative position of two objects may make prediction easier while destroying information required for collision avoidance. Feature prediction does not discover the correct invariances independently of its target representation; it learns to match the invariances already expressed by that representation.

This is why low prediction error is not itself evidence of a good state. It establishes that one branch can match selected outputs of another branch on the sampled training distribution. To establish that the features are useful, they must be tested on predictions or decisions that were not built into the pretraining loss.

4.1.4. A score is not yet a probability or a task objective

It is useful to describe the feature discrepancy as a compatibility score. Define

$$E(x_C, q_T, z_T) = \ell(g_\phi(f_\theta(x_C), q_T), z_T)\,.$$

A smaller value means that the candidate target feature is more compatible with the context and query under the learned model. Averaging this score over training examples gives a prediction loss.

The score is not automatically a probability. To turn an energy E into a normalized conditional density, one must choose a base measure and a partition function:

$$p(z_T \mid x_C, q_T) = \frac{\exp[-E(x_C, q_T, z_T)]}{\int \exp[-E(x_C, q_T, z)]\, d\nu(z)}.$$

For a high-dimensional continuous feature space, the normalizing integral may be intractable. A model can therefore use an energy for ranking candidates without having a calibrated probability distribution over them.

A task cost is different again. A planner may care about reaching a goal, avoiding collision or limiting force. The prediction loss teaches the model to match observed target features; a task cost evaluates whether an imagined outcome is desirable. Conflating these quantities makes a learned model appear more capable than its objective warrants.

This distinction will become important when the chapter moves from masked prediction to robot control. A predictor can estimate a future feature. A planner needs a way to compare futures. The two operations are connected, but they are not the same.

Figure 4.1. The prediction target is a representation, not a reconstructed patch.

Description: A context encoder processes the visible part of an image or clip. A separate target pathway encodes the complete observation and supplies features at selected positions. The predictor receives the context features and target-location query, then estimates the selected target features.

4.2. What exactly is being predicted?

The notation of a JEPA is compact, but the statistical problem is determined by the variables supplied to it. A method must say what the context contains, what the target contains, how the target location is specified and how target examples are sampled. If these choices are left implicit, two objectives that use the same encoder and loss can be mistaken for the same task even when they condition on different information.

4.2.1. The joint distribution defines the task

Let X denote the available observation, C the context-selection variable, T the target-selection variable and Q the query supplied to the predictor. Let Y_T denote the target-side observation at the selected positions. The data pipeline induces a joint distribution

$$p_{\mathcal{D}}(X, C, T, Q, Y_T).$$

The predictor does not learn an abstract relation between context and target in isolation. It learns under this distribution. A different context sampler changes what can be observed. A different target sampler changes what must be inferred. A different temporal stride changes the time interval separating context and target. A different query changes which part of the target representation is requested.

For example, predicting a small block adjacent to a visible block can be solved using local continuity. Predicting a distant object region may require information distributed across the scene. Predicting a feature from a later video frame requires temporal evidence. Predicting the consequence of a robot command requires the command itself to be part of the condition.

It is therefore more accurate to describe a JEPA as learning a conditional prediction problem than as simply learning to ``predict missing patches''. The patches are one way to instantiate the variables.

4.2.2. The mask is part of the conditional distribution

Let M denote the sampled mask. It determines the visible input X_C and the target positions T. The predictor is trained against a distribution of the form

$$p(Z_T \mid X_C, Q_T, M).$$

A mask is not a neutral deletion operation. It determines which evidence remains available and which distinctions the model must infer. If a target can be recovered from a tiny adjacent region, the model may rely on local texture. If the target is far from the visible region, it must combine more global context. If the target is hidden across several frames, the model must use temporal information that survives elsewhere in the clip.

There is no universally correct mask independent of the task. A mask that makes targets almost trivial can reward shortcuts. A mask that removes nearly all useful evidence can make the conditional target highly uncertain. The objective is shaped by the range of conditional problems sampled during training, not merely by the choice of neural architecture.

The query matters for the same reason. A target feature is indexed by a location. If the predictor is not told which location it should estimate, it must either produce all target positions or infer the

requested position from another convention. Supplying explicit position information makes the prediction interface well-defined. I-JEPA uses spatial target queries, while V-JEPA extends the query to spatiotemporal positions (Assran et al., 2023; Bardes et al., 2024).

4.2.3. What does squared-error prediction return?

Suppose the target feature is a random vector H, and the predictor must output one deterministic vector a after observing context $X_C = x$ and query $Q = q$. Under squared error, its ideal output solves

$$a^*(x, q) = \arg\min_a \mathbb{E}\left[\|H - a\|_2^2 \mid X_C = x, Q = q\right].$$

Let

$$\mu(x, q) = \mathbb{E}[H \mid X_C = x, Q = q].$$

Write $H - a = (H - \mu) + (\mu - a)$. Expanding the squared norm gives

$$\begin{aligned}\mathbb{E}[\|H - a\|_2^2 \mid x, q] = \mathbb{E}[\|H - \mu\|_2^2 \mid x, q] \\ + \|\mu - a\|_2^2 \\ + 2\,\mathbb{E}[(H - \mu)^\top (\mu - a) \mid x, q].\end{aligned}$$

The final term is zero because $\mathbb{E}[H - \mu \mid x, q=0]$. The first term does not depend on a. The minimum is therefore attained at

$$a^*(x, q) = \mathbb{E}[H \mid X_C = x, Q = q].$$

A deterministic squared-error predictor estimates the conditional mean of the target features. It does not necessarily select one of the possible target outcomes.

This matters when the conditional target is multimodal. Imagine a scene in which a ball may pass to the left or right of an obstacle. If the two outcomes are represented by distant feature vectors and occur with similar probability, their mean may lie between them. The mean can minimize squared error without corresponding to a plausible physical outcome. The predictor has averaged uncertainty rather than represented the alternatives.

For absolute error, the scalar minimizer is a conditional median. Applied coordinate-wise to a vector, an L_1 loss produces coordinate-wise conditional medians, which may also combine components from different modes. A point-estimation loss therefore does not, by itself, represent a full distribution over possible futures.

4.2.4. When one prediction is not enough

If the task requires multiple alternatives, the model needs an output or inference mechanism capable of representing them. One option is to introduce a latent variable U that selects a hypothesis:

$$\hat{Z}_T = g_\phi(Z_C, Q_T, U).$$

Another is to predict parameters of a conditional distribution. These extensions change the prediction problem: training must specify how latent hypotheses or distributional outputs are supervised and how uncertainty is evaluated.

A latent variable also introduces a possible shortcut. If it can carry the target feature directly, the predictor may ignore the context. The model must constrain how the latent is sampled or inferred, or otherwise prevent it from becoming an unrestricted channel for the answer. A latent variable should represent uncertainty that remains after conditioning on context, not leak the target into the predictor.

Not every JEPA requires a multimodal output. For a target that is nearly determined by context, a deterministic feature estimate can be adequate. The point is to match the output parameterization to the conditional uncertainty of the task. A low pointwise error on an averaged feature is not evidence that alternative outcomes have been represented.

4.2.5. The target branch is not an independent copy of the target crop

In I-JEPA and V-JEPA, the target encoder processes the complete image or video before target features are selected. The context encoder receives only visible tokens. This asymmetry is central: the target feature is contextualized by the target encoder's view of the full observation, while the predictor must estimate it from a restricted context.

The target is therefore not generally the output of encoding an isolated crop. Encoding a patch in isolation could discard its relation to the surrounding scene, while the full-image target feature can include global context. The target pathway defines what the predictor is being asked to approximate.

A second design choice is how the target parameters change. In the original I-JEPA and V-JEPA recipes, the target encoder is updated as an exponential moving average (EMA) of the online context encoder. The target tensor is detached during the prediction loss; the target parameters change through the EMA update rather than through backpropagation. These are two distinct operations. Stop-gradient specifies the derivative path. EMA specifies how the target network evolves between training steps.

Neither choice is a universal requirement for every method called JEPA. LeJEPA, for example,

uses direct view agreement with an explicit distributional regularizer rather than the same teacher-predictor arrangement. What matters is that the optimization has a well-defined target and a reason the target cannot simply follow the predictor's current output.

4.2.6. A predictor is useful when the target contains withheld information

If two branches receive equivalent views and the objective only asks their embeddings to agree, a direct consistency loss may be sufficient. A predictor becomes important when the target contains information intentionally absent from the context, or when the target representation is defined by a distinct branch. The predictor must then map from what is available to what is requested.

This distinction prevents a common overgeneralization: a predictor is not a decorative network inserted into every self-supervised system. Its role follows from the information flow. If the target can be obtained from the context by a simple agreement relation, a separate predictor may be unnecessary. If the target depends on masked locations, later observations or an action that is not present in the context, the prediction interface must account for that missing condition.

Figure 4.2. The mask and query define the conditional prediction problem.

Description: A single observation is divided into visible context and withheld target positions. The query identifies the target locations, while the mask sampler determines which evidence remains available. Nearby, distant, temporal and action-conditioned targets correspond to different conditional distributions.

Figure 4.3. A conditional mean can fall between plausible target modes.

Description: Two separated clusters represent alternative target features compatible with the same context. A squared-error predictor returns their probability-weighted mean, which may lie between the clusters rather than on either plausible outcome.

4.3. I-JEPA: spatial prediction from an image

I-JEPA is a useful first concrete example because its prediction problem can be drawn on a single image. It asks whether features at withheld image locations can be estimated from a spatially distributed context. The method does not need an image decoder to reconstruct the missing pixels. It uses a target encoder to define the feature representation and a predictor to estimate selected target features from visible context.

The important part is not simply that some patches are hidden. The context and target masks are chosen to make the predictor use information that extends beyond the immediate neighbourhood of a target.

4.3.1. From an image to tokens

Let an image be divided into non-overlapping patches. A Vision Transformer maps these patches, together with positional embeddings, into a sequence of tokens. We can index the token at spatial location r by x_r.

The context mask selects a set C of visible positions. The target mask selects one or more disjoint or partially overlapping regions $T_1, \ldots, T_M$. The context encoder receives the visible tokens:

$$Z_C = f_\theta(X_C).$$

The target encoder receives the full image and produces a feature map:

$$H = f_{\bar{\theta}}(X).$$

The target features for block T_m are then selected:

$$H_{T_m} = [H]_{T_m}.$$

This order matters. The target encoder has processed the full image before the target features are read out. Its representation at a target location can therefore depend on surrounding content. The predictor is not asked to reconstruct an isolated patch representation; it is asked to estimate the feature produced by the full-image target pathway.

4.3.2. A context block and several target blocks

I-JEPA uses a large context region and multiple target blocks. The context is spatially distributed rather than being a small crop adjacent to every target. The target blocks are selected at locations that are not supplied as visible input to the context encoder.

The released ImageNet training configuration uses one context block and four target blocks. Its context scale is approximately 0.85 to 1.0 of the image area, while each target block is sampled at a smaller scale, approximately 0.15 to 0.2, with aspect ratios sampled between 0.75 and 1.5. These values describe that recipe, not a definition of I-JEPA. The method is the broader context-to-target feature-prediction construction; mask count and geometry are hyperparameters that alter the sampled prediction problem.

The multiple targets matter. A context may contain enough information to predict several spatially separated regions. Training against multiple blocks discourages the predictor from treating one adjacent boundary as the only useful source of evidence. It does not guarantee that the model learns object-level semantics, but it changes what evidence is useful for reducing the loss.

The context and target regions also need a clear overlap rule. In the standard disjoint setting, target

positions are removed from the context input. If overlap is allowed, a target feature may be partly visible to the context encoder, making the prediction easier for reasons unrelated to the intended inference. The mask construction is therefore part of the experimental method, not an incidental preprocessing detail.

4.3.3. Target positions are queries

The predictor must know which target positions it is expected to estimate. I-JEPA supplies positional information for the target blocks. Conceptually, the predictor receives the context features and a set of target-position queries:

$$\widehat{H}_{T_m} = g_\phi(Z_C, Q_{T_m}).$$

The predictor is not given the target pixels or the target encoder's feature values. It receives the spatial coordinates of the requested features and must infer their values from context.

This separates two roles that can otherwise be confused. The mask says which input tokens are withheld. The query says where the target features are located. The predictor uses the query to place its estimates in the target feature map. If the positions were omitted, the model would not know which spatial features the loss compares.

The predictor in the released architecture is narrower than the main encoder. This is a computational choice, not the conceptual reason prediction works. The predictor provides a mapping from context features to target features; its width and depth determine capacity and cost.

4.3.4. The loss and the target update

At the level of the paper's objective, I-JEPA averages a feature-distance loss over the selected target positions and target blocks. The target branch is updated by an EMA of the online encoder. The exact training implementation includes feature normalization and uses smooth-L_1 regression in the released training script; this is worth distinguishing from the paper's compact equation, which presents an L_2-style feature distance.

A simplified expression for the implementation is

$$\mathcal{L}_{\text{I-JEPA}} = \frac{1}{M} \sum_{m=1}^{M} \text{SmoothL1}\left(\widehat{H}_{T_m}, \text{sg}(\text{LN}(H_{T_m}))\right),$$

where LN is feature-wise layer normalization and sg denotes stop-gradient. The notation suppresses the average over batch and spatial positions.

The online context encoder and predictor are updated by backpropagation. The target encoder

receives no gradient from this loss. Its parameters follow the online encoder through

$$\bar{\theta} \leftarrow m\bar{\theta} + (1 - m)\theta,$$

with a momentum coefficient m scheduled during training. This update makes the target evolve more slowly than the online model. It reduces the tendency for the target to move immediately in response to the predictor's current error, but it is not, by itself, a theorem that rules out every degenerate solution.

The distinction between paper equation and released implementation is not cosmetic. A mathematical description should state the objective being analysed; an implementation description should state the numerical operations actually used. When those differ, report both rather than silently treating one as the other. The target/context paths and EMA update are described in the I-JEPA paper; the normalization and smooth-L1 details above refer to the released training code (Assran et al., 2023; official implementation).

4.3.5. Why spatial prediction can produce useful features

A target block is easier to predict when the context retains information related to its content. That information can include object identity, scene layout, shape, texture and spatial relationships. A large distributed context allows the predictor to combine evidence from more than the target's immediate neighbourhood.

The target encoder also changes what counts as a successful prediction. Because it processes the full image, the feature at one target location can summarize both local content and relationships to the rest of the scene. The predictor learns to estimate this contextualized feature from the visible context.

The method's evidence comes from downstream evaluations, not just from a decreasing pretraining loss. The I-JEPA paper evaluates frozen or lightly adapted representations on tasks that include image classification, low-shot classification, object counting and depth estimation. These tests show that spatial feature prediction can yield representations useful for several visual tasks. They do not show that the learned features form a physically sufficient state for control.

The target mask ablations are also informative. Large target regions and a sufficiently informative context performed better than alternatives in the reported evaluations. This supports the idea that mask geometry controls the information the model must use. It does not establish one universally optimal mask across datasets or downstream tasks.

4.3.6. What I-JEPA does not learn

I-JEPA has no temporal index in its prediction target. It does not have to distinguish two histories

that lead to different motion, because the target and context belong to one image. It has no action variable, so it cannot answer what would happen under a selected robot command. It is not trained through recursive prediction of its own estimates, so its loss does not test long-horizon consistency.

These are boundaries of the chosen problem, not defects in the method. I-JEPA demonstrates that useful image representations can be learned by predicting target features from spatial context without reconstructing the image. To ask about motion, the target must extend through time. To ask about controlled consequences, the action must become part of the condition.

Figure 4.4. I-JEPA predicts contextualized features at hidden image locations.

Description: A large context region supplies visible patch tokens to the online encoder. The target encoder processes the full image, after which several target blocks are selected. Positional queries tell the predictor which target features to estimate; no target pixels or target feature values enter the predictor.

4.4. V-JEPA: extending prediction through time

An image describes a scene at one instant. It does not directly specify how that scene changes. A sequence adds temporal evidence: an object may move, a hand may make contact, or a tool may alter another object's state. V-JEPA extends feature prediction from a spatial grid to a spatiotemporal grid. It asks the model to predict selected video features from the remainder of the clip.

This is a more demanding representation-learning problem than image masking, but it is not automatically a causal or action-conditioned dynamics model. That distinction depends on which frames are visible and whether the predictor receives an explicit action.

4.4.1. Video tokens have spatial and temporal coordinates

Let a video clip contain frames $X = (x_1, \ldots, x_T)$. A video encoder converts short spatiotemporal volumes, or tubelets, into tokens. Each token can be indexed by

$$r = (t, h, w),$$

where t is temporal position and h, w are spatial coordinates.

A context mask selects visible tokens C, and a target mask selects positions T. The online encoder processes the visible subset, while the target encoder processes the complete clip:

$$Z_C = f_\theta(X_C), \qquad H_T = [f_{\bar{\theta}}(X)]_T.$$

The predictor uses the context representation and the spatiotemporal target query to estimate $\widehat{H}_T$.

The loss compares this estimate with H_T.

The additional time coordinate changes what can be predicted. A target at the same spatial position in a later frame may depend on motion. A target at a different position may depend on object displacement or interaction. A target extended over several frames may require the predictor to estimate a temporally coherent feature sequence.

4.4.2. Why the target is masked across time

Video contains strong redundancy. Nearby frames often show nearly the same background and similar object appearance. If a target patch is hidden at one frame but visible at adjacent frames, the model may infer it from a near-duplicate rather than learn a useful relation across the clip.

The original V-JEPA uses 3D multi-block masking. It samples spatially contiguous blocks and extends each selected spatial region through the temporal dimension. Its reported short-range configuration takes the union of eight target blocks with spatial scale 0.15; its long-range configuration takes the union of two blocks with spatial scale 0.7. Block aspect ratios are sampled between 0.75 and 1.5. The union of blocks produces an average masking ratio near 90% in the reported setup. These settings define the experiments in the paper; they are not universal constants of video JEPA.

The spatial mask is repeated across the clip's temporal dimension. The context encoder therefore cannot inspect the same spatial region in neighbouring frames. It must use other spatial regions and the remaining temporal evidence to predict the target tube.

Mask scale changes the inference problem. Smaller blocks can test whether local and moderately distributed context predicts withheld content. Larger blocks remove more of the scene and require broader evidence. Combining the two exposes the predictor to more than one spatial scale. It still does not force a strictly past-to-future prediction, because the visible context may include frames both before and after a target time.

4.4.3. The original objective predicts target features

The original V-JEPA predicts the target encoder's features, not RGB pixels. Its paper formulates the feature regression with an L_1 loss:

$$\mathcal{L}_{\text{V-JEPA}} = \frac{1}{|T|D} \sum_{r \in T} \left\| \widehat{h}_r - h_r \right\|_1,$$

where D is feature dimension and h_r is the target feature at spatiotemporal position r. The target feature is supplied by the slowly updated target encoder and detached from the prediction-loss gradient.

This choice has a statistical interpretation. For a scalar target H, the constant a minimizing $\mathbb{E}[|H - a| \mid C]$ is a conditional median. Squared error instead selects the conditional mean. A coordinate-wise vector L_1 loss therefore favours coordinate-wise conditional medians, with possible non-uniqueness. This does not mean V-JEPA explicitly models a multimodal probability distribution. It means the chosen pointwise loss has a different optimum from squared-error regression.

The target encoder is updated through an EMA of the online encoder, as in I-JEPA. The target branch is not updated by backpropagation through the feature loss. This creates a moving target that changes more slowly than the online context encoder. It is an optimization design, not a universal guarantee against collapse.

4.4.4. Temporal evidence is not the same as an action

A video model can exploit temporal regularities. If a hand approaches a cup, the next observed frames may often show a grasp. The model can learn that this visual pattern predicts a particular feature configuration. Yet it has not necessarily learned the effect of a chosen intervention.

The distinction is between two conditional distributions:

$$p(Z_{t+1} \mid X_{1:t}) \qquad \text{and} \qquad p(Z_{t+1} \mid X_{1:t}, A_t).$$

The first describes future features under the action distribution present in the observed trajectories. The second conditions on a specified action A_t. If the action is omitted, the predictor cannot be asked to compare the consequences of two alternative commands while holding the history fixed.

Temporal order alone also does not guarantee that context is past-only. The original V-JEPA paper studies both full-clip multi-block masking and more restrictive causal-prefix variants. The standard multi-block objective can expose the context encoder to frames later than a selected target position. It is therefore not a causal next-frame objective. The paper's ablations show that the full-clip context setting can be effective for learning video representations; they do not turn it into an intervention model.

This distinction should be maintained even when an evaluation is called action anticipation. Predicting a likely human action label from a video is a semantic forecast. Predicting the physical consequences of a robot command requires the command itself to be an input and requires training data that connects actions to subsequent observations.

4.4.5. What V-JEPA demonstrates

The original V-JEPA results show that masked spatiotemporal feature prediction can produce useful frozen video representations. Evaluation on action recognition and video understanding

tasks indicates that features learned from temporal context transfer to downstream tasks. The paper also studies mask design and the contribution of temporal information.

Those results support a limited but important conclusion: it is possible to learn video features by predicting withheld spatiotemporal representations without reconstructing every pixel. They do not demonstrate a complete dynamics model, a calibrated distribution over futures or successful planning under candidate actions.

A representation can be useful for recognizing motion while omitting variables needed for physical prediction. It can support classification without being Markov. It can predict features of observed clips without remaining accurate when its own predictions are fed back as inputs. Each stronger claim needs a different evaluation.

The progression from I-JEPA to V-JEPA therefore adds temporal evidence but not control. To predict what follows a chosen action, the action must enter the conditioning information. V-JEPA 2 makes that transition explicit by retaining a pretrained visual encoder and training a separate action-conditioned predictor on robot trajectories.

Figure 4.5. A spatiotemporal mask with full-clip context is not inherently causal.

Description: A spatial target region is withheld through several frames, while the context contains the remaining tokens from the clip. Some context may lie after a target time. A separate causal-prefix design would restrict context to earlier frames only.

Figure 4.6. Masked video prediction and action-conditioned prediction answer different questions.

Description: The masked-video path estimates withheld features from visible video context. The controlled path receives an explicit action alongside the current state and predicts a resulting feature or state. Changing the action while holding context fixed must be able to change the controlled prediction.

4.5. Why prediction agreement can still collapse

The JEPA objective asks a predictor to match target features. It does not automatically require those features to vary with the input. This is the central difficulty of non-contrastive feature prediction: agreement is a relation between outputs, whereas information is a property of how those outputs depend on the observations.

The distinction should be derived from the loss rather than treated as an optimizer failure.

4.5.1. The constant solution

Consider the feature-prediction loss

$$\mathcal{L} = \mathbb{E}\left[\left\|g_\phi(f_\theta(X_C), Q_T) - f_{\bar{\theta}}(X)_T\right\|_2^2\right].$$

Suppose the context encoder and target encoder both return the same constant vector c for every input, and the predictor returns c for every query. The loss becomes

$$\mathcal{L} = \mathbb{E}[\|c - c\|_2^2] = 0.$$

Since squared distance is nonnegative, this is a global minimum of the stated objective. The model has satisfied the training criterion while discarding all information about the input.

The argument does not depend on the value of c. It can be zero or any fixed vector. It also does not depend on whether the loss is squared error, absolute error or another discrepancy that is zero when its arguments are equal. If both branches are allowed to emit the same constant, agreement alone cannot distinguish this solution from an informative one.

This is an identifiability problem. The loss specifies a relation the outputs should satisfy, but it does not identify which solutions preserve useful information. Optimizing the objective cannot supply a requirement that the objective never expresses.

4.5.2. A target update changes dynamics, not the mathematical requirement

I-JEPA and the original V-JEPA use an online context encoder and an EMA target encoder. The target parameters evolve according to

$$\bar{\theta}_{k+1} = m_k \bar{\theta}_k + (1 - m_k)\theta_{k+1}, \qquad 0 \le m_k < 1.$$

The target is a smoothed history of the online parameters. A large momentum coefficient makes the target change slowly. Stop-gradient prevents the prediction loss from directly differentiating through the target features; the EMA update is a separate mechanism that changes target parameters.

These choices can stabilize optimization. They do not remove the constant solution from the objective. If both branches are constant and the predictor matches them, the prediction loss remains zero. Whether training enters or escapes such a solution depends on initialization, data, architecture, normalization, optimizer and the complete training dynamics.

It is therefore incorrect to say that an EMA teacher mathematically prevents collapse in every JEPA. The teacher changes the moving-target problem faced by the online network. It does not, by itself, require the representation to encode any particular variable.

4.5.3. Collapse can be partial

Complete collapse maps every input to the same feature. A less obvious form occurs when the representation varies only in a low-dimensional subspace.

Let $Z \in \mathbb{R}^{N \times D}$ be a batch of centred feature vectors and define the empirical covariance

$$\hat{\Sigma} = \frac{1}{N-1} Z^\top Z.$$

If the centred features span a subspace of dimension $r < D$, then

$$\operatorname{rank}(\hat{\Sigma}) \leq r.$$

At least $D - r$ eigenvalues are zero. The representation uses only a subset of its nominal dimensions. This is dimensional collapse: the representation is not constant, but multiple coordinates may carry redundant combinations of the same few directions.

Marginal variance alone does not rule this out. Two coordinates can both vary while being nearly identical or exact linear combinations. A variance floor can prevent each coordinate from becoming constant, yet still permit strong correlations. A covariance penalty can reduce linear redundancy, but it need not give every dimension useful scale unless scale is constrained separately.

Nor does full rank establish semantic quality. A representation can fill every direction with nuisance variation and still discard the location, velocity or contact information a downstream model needs.

4.5.4. Agreement and representation quality are separate constraints

It is useful to separate the learning objective into two responsibilities:

$$\mathcal{L} = \mathcal{L}_{\text{agreement}} + \lambda \mathcal{R}(Z).$$

The first term specifies which examples or views should produce compatible outputs. The second constrains the distribution or geometry of the embeddings. This decomposition does not prescribe one universal regularizer. It makes explicit that the two terms answer different questions.

Agreement asks: which outputs should be close?

A representation constraint asks: what kinds of output distributions should be allowed?

A negative-sample objective couples examples by asking a positive pair to be more similar than

candidate negatives. A variance penalty asks that each coordinate retain a minimum amount of spread. A covariance penalty discourages linear redundancy. A distributional penalty compares the whole embedding distribution, or a sketch of it, with a chosen reference. These mechanisms are not interchangeable because they constrain different mathematical objects. The contrastive, balanced-assignment and variance/covariance examples connect to methods developed earlier in Chapter 3; SIGReg and VISReg are treated here because they directly constrain the embedding distribution (Chen et al., 2020; Caron et al., 2020; Grill et al., 2020; Bardes et al., 2022; Balestriero and LeCun, 2025; Wu et al., 2026).

Chapter 3 already developed contrastive and non-contrastive mechanisms in detail. Here the relevant point is narrower: feature prediction needs an explanation for why the target features do not all become identical and why non-identical features retain the right information.

4.5.5. A predictor is not an anti-collapse proof

A predictor can make the two branches asymmetric. It can also make the optimization problem easier by learning a mapping from context features to target features. But adding a predictor does not logically eliminate every collapsed solution. If both encoders emit a constant and the predictor emits that same constant, the feature loss can still be zero.

The same caution applies to stop-gradient, normalization and architectural asymmetry. Each changes the computation or optimization path. None should be described as a universal semantic guarantee.

There are methods that rely on explicit distributional constraints instead. LeJEPA uses view agreement together with SIGReg, which regularizes each view’s embedding distribution toward an isotropic Gaussian. VISReg uses an invariance term together with separate scale, shape and centring constraints. Their objectives differ from the EMA-predictor recipes of I-JEPA and V-JEPA. The reason to introduce them is not to catalogue another pair of methods. It is to examine what representation geometry is being requested and what statistical argument supports that request.

4.5.6. Non-collapse is only the first test

A representation can have nonzero variance, full-rank covariance and a roughly Gaussian marginal distribution while still failing to preserve a task-relevant distinction. A video encoder may vary strongly with appearance but ignore object velocity. A robot representation may encode gripper position but discard the relation between the gripper and a nearby object.

Non-collapse rules out one class of trivial solutions. It does not establish predictive sufficiency. The next question is more demanding: for a specified downstream target and action space, does the representation preserve all distinctions in the history that change the relevant future distribution?

Figure 4.7. Complete collapse and dimensional collapse are different failures.

Description: In complete collapse, all observations map to one point. In dimensional collapse, the embedding cloud varies but lies in a lower-dimensional subspace, leaving several directions unused or redundant. A nonzero variance in every coordinate does not by itself rule out the second failure.

4.6. What should a predictive representation preserve?

A non-collapsed representation can still be a poor state. The relevant question is not whether the embedding varies, but whether it distinguishes histories that lead to different predictions or decisions. This makes representation quality a task-relative property.

4.6.1. Predictive sufficiency is defined relative to a target

Let X denote the observation history available to an agent, A an action or action sequence, Y a future quantity of interest, and

$$Z = f_\theta(X)$$

the learned representation. For the chosen prediction problem, Z is sufficient when

$$p(Y \mid X, A) = p(Y \mid Z, A)$$

almost surely under the data distribution.

The equality says that once the representation and action are known, the original history provides no additional information about the target distribution. This is a statistical definition, not a claim that Z equals the simulator's physical state.

The target Y, action set, prediction horizon and data distribution must be specified. A representation can be sufficient for object classification but insufficient for contact prediction. It can be sufficient for a short visual forecast but insufficient for a long physical rollout. It can be sufficient for passive future prediction yet insufficient for predicting the effect of a candidate action.

The action appears on both sides because a controller needs to know whether two histories respond differently to the same command. If x_1 and x_2 map to the same Z, but the same action produces different future distributions, then the representation has merged histories the controller must distinguish.

4.6.2. Predictive equivalence between histories

The condition can be understood as an equivalence relation. Fix the family of targets and admissible actions relevant to the task. Two histories x_1 and x_2 are predictively equivalent when

$$x_1 \sim x_2 \quad \Longleftrightarrow \quad p(Y \mid X = x_1, A = a) = p(Y \mid X = x_2, A = a)$$

for every relevant target Y and action a.

A minimal sufficient representation would assign the same code to histories within one equivalence class and different codes to histories whose relevant conditional futures differ. A learned encoder is not guaranteed to recover these classes. It is trained through a proxy target, and the target encoder may already identify some observations as equivalent.

Consider two images of the same cup under different lighting. If the lighting change leaves every downstream prediction unchanged, treating the images as equivalent is useful. Now consider two images with the cup at different distances from a gripper. If the gripper will move under the same command, those positions may lead to different collision or contact outcomes. Treating them as equivalent could be destructive.

The correct invariances are therefore determined by the task, not by visual similarity alone. This task-relative view of a predictive state is also central to the autonomous-machine-intelligence proposal (LeCun, 2022).

4.6.3. The target encoder decides which differences count

Suppose two target observations satisfy

$$f_{\bar{\theta}}(y_1) = f_{\bar{\theta}}(y_2).$$

A predictor matching this target feature has no direct reason to distinguish y_1 from y_2. The target encoder has made them equivalent for the loss.

This can be exactly what is wanted. The target may remove small lighting differences, sensor noise or background texture. But if it also removes object position, relative orientation or a contact configuration required downstream, the prediction loss cannot recover that distinction by itself.

The target encoder is therefore part of the task specification. Its architecture, training data, augmentations and previous objectives define the geometry in which prediction errors are measured. A dense feature map may preserve spatial correspondence better than a pooled vector, but density alone does not guarantee that the right spatial information is present.

This also explains why a representation benchmark is not a universal measure of state quality. A

feature can perform well on an image classification benchmark while discarding information needed for manipulation. A probe only measures the information that its target and readout ask for.

4.6.4. What a probe actually measures

A frozen representation is often evaluated by fitting a predictor on top of it. The predictor class determines the claim supported by the result.

A linear probe asks whether the target is accessible through a linear map. A nonlinear probe asks whether a more expressive function can extract it. A nearest-neighbour or kernel estimator tests whether local geometry supports prediction. A closed-loop control evaluation tests whether the complete model and controller can use the representation to select actions in an environment.

These tests are not interchangeable. A weak linear probe does not prove that the information is absent; it may be encoded in a form that a linear map cannot extract. A strong nonlinear probe does not show that the information is easy to use for planning. Successful planning is evidence about the complete system, including the transition predictor, cost, search procedure and feedback loop.

A useful evaluation should therefore match the readout to the claim. Representation quality, feature prediction, action-conditioned prediction and closed-loop control should be reported separately.

4.6.5. Covariance describes only second-order geometry

Arrange N centred embeddings in a matrix $Z \in \mathbb{R}^{N \times D}$. The empirical covariance is

$$\widehat{\Sigma} = \frac{1}{N-1} Z^{\top} Z.$$

Its diagonal entries are coordinate variances. Its off-diagonal entries measure pairwise linear co-variation. Its eigenvectors give principal directions of variation, while its eigenvalues quantify variance along those directions.

If the eigenvalues are equal, the covariance is isotropic:

$$\widehat{\Sigma} = \sigma^2 I_D.$$

If a few eigenvalues dominate while others are small, the embedding cloud is anisotropic. The representation varies more in some directions than others. This can matter for downstream estimation: a small eigenvalue means the dataset contains little variation along that feature direction.

Covariance is not the full distribution. Two distributions can have identical means and covariance while differing in skewness, multimodality, tails or support. Zero covariance does not imply independence, and a full-rank covariance matrix does not imply semantic sufficiency.

The covariance spectrum is nevertheless a useful starting point because it determines the conditioning of simple linear readouts. The next section derives that dependence for ridge regression and ordinary least squares. This gives a concrete reason to care about weak feature directions before considering more complicated nonlinear estimators.

Figure 4.8. Predictive equivalence is defined by downstream futures, not visual similarity.

Description: Two histories that differ only in nuisance appearance are grouped into one predictive-equivalence class when their target distributions agree. Histories that look similar but respond differently to the same action must remain distinguishable.

Figure 4.9. Equal total variance can hide very different covariance spectra.

Description: An isotropic embedding distributes variance equally across directions. An anisotropic embedding concentrates variance in a few directions and leaves weak directions with small eigenvalues. Both clouds can have the same covariance trace, but their downstream estimation problems are differently conditioned.

4.7. Why the covariance spectrum affects linear readouts

The covariance spectrum matters because a downstream model must estimate its parameters from a finite sample. If the representation varies strongly in some directions and barely varies in others, the regression problem is poorly conditioned. This is not a statement that isotropic embeddings are best for every task. It is a result about specific estimators under explicit assumptions.

4.7.1. The frozen-feature regression problem

Suppose an encoder is trained and then frozen. A downstream task provides pairs (z_i, y_i), where $z_i \in \mathbb{R}^D$ is a representation and y_i is a scalar target. A linear readout models

$$y_i = z_i^\top \beta^\star + \varepsilon_i,$$

where $\beta^\star$ is the population coefficient and ε_i is zero-mean observation noise.

Let the centred design matrix be $X \in \mathbb{R}^{N \times D}$, with rows $z_i^\top$, and let $\widehat{\Sigma} = X^\top X / N$ be the sample covariance. We focus on the population covariance Σ that the sample estimates. Assume Σ is positive definite and has eigen-decomposition

$$\Sigma = U \Lambda U^\top, \qquad \Lambda = \mathbf{diag}(\lambda_1, \ldots, \lambda_D),$$

with all $\lambda_j > 0$.

The eigenvalues describe how much variation is available to estimate a coefficient along each principal direction. If an eigenvalue is small, many different coefficients along that direction can produce similar predictions on the finite sample. The readout must estimate that coefficient from weak evidence.

4.7.2. Ridge shrinks weak directions more strongly

For a normalized ridge objective

$$\widehat{\beta}_\lambda = \arg\min_\beta \left[\frac{1}{N} \|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2 \right],$$

the solution is

$$\widehat{\beta}_\lambda = (\widehat{\Sigma} + \lambda I)^{-1} \frac{X^\top y}{N}.$$

Under the population approximation $\widehat{\Sigma} \approx \Sigma$, the expected estimate is

$$\mathbb{E}[\widehat{\beta}_\lambda \mid \Sigma] \approx (\Sigma + \lambda I)^{-1} \Sigma \beta^\star.$$

In the eigenbasis, each coefficient is multiplied by

$$\frac{\lambda_j}{\lambda_j + \lambda}.$$

The corresponding bias component is

$$\mathrm{Bias}_j = -\frac{\lambda}{\lambda_j + \lambda} \beta_j^\star.$$

When λ_j is large relative to λ, the coefficient is only mildly shrunk. When λ_j is small, the shrinkage is strong. If the downstream task places weight on a weak feature direction, ridge may suppress that task-relevant coefficient.

Now hold the total feature variance fixed:

$$\mathrm{tr}(\Sigma) = \sum_{j=1}^{D} \lambda_j = \tau.$$

The average eigenvalue is $\bar{\lambda} = \tau/D$. The largest possible worst-direction ridge bias is controlled by the smallest eigenvalue. Since

$$\lambda_{\min} \le \bar{\lambda},$$

an anisotropic covariance has at least one direction no stronger than the isotropic covariance with eigenvalues all equal to $\bar{\lambda}$. A task whose coefficient lies in that weak direction suffers at least as much shrinkage as in the isotropic case, and more if $\lambda_{\min} < \bar{\lambda}$.

This is a worst-case statement over task directions. It does not say that every downstream task performs worse with anisotropic features. A task aligned with a high-variance direction may be estimated well. The result says that severe anisotropy creates weak directions in which a linear task can become difficult.

4.7.3. Ordinary least squares exposes the variance penalty

Set $\lambda = 0$. Under full column rank, ordinary least squares gives

$$\widehat{\beta}_{\mathrm{OLS}} = (X^\top X)^{-1} X^\top y.$$

Assume the classical homoskedastic model

$$\mathrm{Cov}(\varepsilon \mid X) = \sigma_\varepsilon^2 I.$$

The conditional coefficient covariance is

$$\mathrm{Cov}(\widehat{\beta}_{\mathrm{OLS}} \mid X) = \sigma_\varepsilon^2 (X^\top X)^{-1}.$$

With $X^\top X \approx N\Sigma$, the expected scale of the coefficient variance is governed by

$$\frac{\sigma_\varepsilon^2}{N} \mathrm{tr}(\Sigma^{-1}) = \frac{\sigma_\varepsilon^2}{N} \sum_{j=1}^{D} \frac{1}{\lambda_j}.$$

Small covariance eigenvalues appear in the denominator. They therefore contribute disproportionately to estimator variance.

At fixed trace $\sum_j \lambda_j = \tau$, convexity of $u \mapsto 1/u$ and Jensen's inequality give

$$\frac{1}{D} \sum_{j=1}^{D} \frac{1}{\lambda_j} \ge \frac{1}{\frac{1}{D} \sum_{j=1}^{D} \lambda_j} = \frac{D}{\tau}.$$

Hence

$$\mathrm{tr}(\Sigma^{-1}) \geq \frac{D^2}{\tau},$$

with equality only when all eigenvalues equal τ/D. Under this fixed-trace, full-rank and homoskedastic linear model, isotropic covariance minimizes the trace of the OLS coefficient covariance. This is the linear downstream-risk result developed in LeJEPA; its scope is the specified estimator and assumptions (Balestriero and LeCun, 2025).

4.7.4. What the calculation establishes

The two results isolate complementary effects. Ridge introduces shrinkage bias, which is strongest in weak feature directions. Ordinary least squares avoids ridge shrinkage but becomes statistically unstable when the design contains weak directions. Equalizing the covariance spectrum removes the weakest-direction disadvantage under the assumptions above.

The result concerns coefficient estimation for a linear readout. It does not prove that isotropic covariance minimizes prediction error for every task, that Gaussian embeddings are always preferable, or that isotropy guarantees physical sufficiency. A representation may be isotropic while encoding the wrong variables. A nonlinear downstream function may depend on distributional structure that covariance does not describe.

The motivation for the next step is therefore limited and precise. If downstream prediction uses nonlinear local estimators, the arrangement of samples within the embedding space matters beyond the covariance spectrum. The local density then enters the estimator's bias.

Figure 4.10. Ridge shrinkage is direction-dependent.

Description: Coefficients aligned with large covariance eigenvalues are retained more strongly than coefficients aligned with weak eigenvalues. At fixed total variance, anisotropy creates a smallest-eigenvalue direction in which shrinkage is worst.

Figure 4.11. OLS variance grows with the inverse covariance spectrum.

Description: Each principal direction contributes in proportion to the reciprocal of its covariance eigenvalue. A narrow direction therefore contributes a large coefficient-variance term, even when the total feature variance is unchanged.

4.8. Nonlinear readouts, local density and Fisher information

A covariance matrix describes global second-order geometry. It does not say where points are concentrated within that geometry. For nonlinear readouts that estimate a target by averaging

nearby examples, this local density matters.

The LeJEPA analysis makes this dependence explicit. Under a specified family of smooth regression functions and local estimators, the leading prediction bias contains the score of the embedding density,

$$s_p(z) = \nabla \log p(z).$$

The expected squared magnitude of this score is the Fisher-information functional

$$J(p) = \mathbb{E}_{Z\sim p}\left[\|\nabla \log p(Z)\|_2^2\right] = \int \|\nabla \log p(z)\|_2^2 p(z)\, dz.$$

This is a property of the embedding distribution. It is not a property of the encoder architecture alone.

4.8.1. Local regression depends on where examples are concentrated

Let $Z \in \mathbb{R}^D$ be a frozen representation with density $p(z)$, and let the downstream regression function be

$$\eta(z) = \mathbb{E}[Y \mid Z = z].$$

A local estimator predicts $\eta(q)$ by averaging labels from examples whose embeddings lie near a query q. In a kernel estimator,

$$\widehat{\eta}_h(q) = \frac{\sum_{i=1}^N K_h(q - Z_i) Y_i}{\sum_{i=1}^N K_h(q - Z_i)}, \qquad K_h(u) = h^{-D} K(u/h).$$

The kernel weights nearby points more heavily. The bandwidth h controls the neighbourhood size. The denominator normalizes the weights.

The estimator is affected by two local properties. The target function may curve within the neighbourhood, so averaging nearby labels can differ from the value at its centre. The embedding density may be asymmetric around the query, so one side of the neighbourhood contributes more examples than the other. The second effect creates a bias even when the target function is locally smooth.

4.8.2. Where the density score enters

For large samples, the conditional expectation of the Nadaraya–Watson estimator is approximately

$$\frac{\int \eta(q+u)p(q+u)K_h(u)\,du}{\int p(q+u)K_h(u)\,du}.$$

Assume a normalized, symmetric kernel with zero first moment and isotropic second moment:

$$\int K(v)\,dv = 1, \qquad \int vK(v)\,dv = 0, \qquad \int vv^\top K(v)\,dv = \mu_2(K) I_D.$$

Expand the regression function and density around q to second order. Odd terms vanish after integration because the kernel is symmetric. The remaining terms include the Laplacian of the regression function and the cross term between its gradient and the density gradient. After division by the denominator, the leading bias is

$$\mathrm{Bias}[\widehat{\eta}_h(q)] = \frac{h^2\mu_2(K)}{2}\left[\Delta\eta(q) + 2\nabla\eta(q)^\top \nabla \log p(q)\right] + o(h^2).$$

The first term, $\Delta\eta(q)$, is target curvature. The second term measures the alignment of target change with the direction in which the embedding density increases. If more training points lie on one side of the query, their labels pull the local average toward that side. If the target function changes along the same direction, the estimator is biased.

The same density-score coupling appears in the leading bias of radius-based k-nearest-neighbour regression. The coefficient differs because a uniform ball and a smooth kernel have different second moments. The presence of $\nabla \log p(q)$ is the common feature.

This expansion requires smoothness. The density must be positive and sufficiently differentiable around the query, the kernel must have finite moments, and the regression function must have the required derivatives. The asymptotic regime also assumes a bandwidth small enough to localize the estimator while leaving enough samples in each neighbourhood. The equation should not be applied to arbitrary discrete or highly singular embedding distributions without checking these conditions.

4.8.3. From pointwise bias to the Fisher-information functional

The downstream risk is evaluated over query points, not at one fixed location. Let queries be drawn from the embedding density p. The integrated squared bias is

$$\mathrm{ISB} = \int \mathrm{Bias}(q)^2 p(q)\,dq.$$

The density-dependent term in the pointwise expansion is $\nabla\eta(q)^\top \nabla \log p(q)$. To understand its

expected contribution, consider a class of downstream functions whose gradients have isotropic second moment:

$$\mathbb{E}_\eta[\nabla\eta(Z)] = 0, \qquad \mathbb{E}_\eta[\nabla\eta(Z)\nabla\eta(Z)^\top] = \tau_g^2 I_D.$$

Then

$$\begin{aligned}
&\mathbb{E}_\eta \mathbb{E}_{Z\sim p}\left[(\nabla\eta(Z)^\top \nabla \log p(Z))^2\right] \\
&\quad = \tau_g^2 \mathbb{E}_{Z\sim p}\left[\|\nabla \log p(Z)\|_2^2\right] \\
&\quad = \tau_g^2 J(p).
\end{aligned}$$

The functional $J(p)$ measures the expected squared score. It is large when the log-density changes sharply over regions carrying substantial probability mass. In a local estimator, this corresponds to density variation that can make one side of a neighbourhood contribute more strongly than the other.

The complete integrated-bias expression also contains a curvature term and a cross term involving the target Hessian and gradient. Those terms do not disappear merely because the score term has been identified. The LeJEPA analysis controls them under its stated regularity and downstream-function assumptions, yielding a density-dependent contribution proportional to $J(p)$ plus bounded terms. The conclusion is not that every nonlinear estimator has risk exactly equal to $J(p)$, but that this functional governs a specific part of the risk bound for the estimator classes analysed.

4.8.4. Gaussianity follows from a fixed-covariance inequality

The Fisher-information functional has a useful lower bound. Let p be a mean-zero density with positive-definite covariance

$$\Sigma = \mathbb{E}[ZZ^\top].$$

Assume sufficient decay and differentiability for integration by parts, and finite Fisher information. Since $p(z)\nabla \log p(z) = \nabla p(z)$,

$$\mathbb{E}[Z\, s_p(Z)^\top] = \int z\, \nabla p(z)^\top dz = -I_D.$$

Now consider the expected outer product

$$\mathbb{E}\left[(s_p(Z) + \Sigma^{-1}Z)(s_p(Z) + \Sigma^{-1}Z)^\top\right].$$

It is positive semidefinite. Expanding it and using the identity above gives

$$\mathbb{E}[s_p(Z)s_p(Z)^\top] - \Sigma^{-1} \succeq 0.$$

Taking the trace yields

$$J(p) \geq \operatorname{tr}(\Sigma^{-1}).$$

Equality requires the score to be linear in z:

$$\nabla \log p(z) = -\Sigma^{-1} z.$$

Integrating this relation gives the Gaussian density with covariance Σ. Thus, among densities with a fixed covariance matrix, the Gaussian uniquely minimizes the Fisher-information functional.

This is the first optimization. It identifies the best shape for a given covariance. It does not yet say which covariance is best.

4.8.5. Isotropy follows from a fixed total-variance constraint

Let the eigenvalues of Σ be $\lambda_1, \dots, \lambda_D > 0$. Then

$$\operatorname{tr}(\Sigma^{-1}) = \sum_{j=1}^{D} \frac{1}{\lambda_j}.$$

Under a fixed trace constraint

$$\sum_{j=1}^{D} \lambda_j = \tau,$$

strict convexity of $u \mapsto 1/u$ gives

$$\sum_{j=1}^{D} \frac{1}{\lambda_j} \geq \frac{D^2}{\tau},$$

with equality only when all eigenvalues equal τ/D. The covariance is then isotropic:

$$\Sigma = \frac{\tau}{D} I_D.$$

Combining the two steps gives the isotropic Gaussian as the unique minimizer of $J(p)$ under a fixed total-variance constraint:

$$p^\star = \mathcal{N}\left(0, \frac{\tau}{D} I_D\right).$$

Gaussianity and isotropy arise from different arguments. The first minimizes Fisher information among distributions with a fixed covariance. The second minimizes the inverse-covariance trace among covariances with fixed total variance.

4.8.6. The scope of the result

The argument supports a principled target distribution for the downstream risk framework analysed in LeJEPA. It is not a theorem that Gaussian embeddings are optimal for every task, every estimator or every control problem. The conclusion depends on the local-estimator approximation, smoothness conditions, downstream-function class and covariance constraint.

It is also important to distinguish isotropy from Gaussianity. A non-Gaussian distribution can have zero mean and identity covariance. Such a distribution satisfies the same second-order constraint but can have a different score field and a larger Fisher-information functional. Covariance regularization alone cannot enforce the full distributional property that appears in this risk argument.

The result motivates the next section. If the target is an isotropic Gaussian, a training objective needs to compare a high-dimensional empirical embedding distribution with that target. SIGReg does so through one-dimensional projections and characteristic functions rather than by estimating a full density.

Figure 4.12. Local prediction bias contains a density-gradient term.

Description: A query lies in an asymmetric neighbourhood of training embeddings. More samples occur on one side, pulling the local estimate toward the denser region. The score $\nabla \log p(q)$ describes this local imbalance.

Figure 4.13. Fisher information and covariance optimality are separate steps.

Description: The first comparison holds covariance fixed and shows that the Gaussian minimizes the score-based Fisher-information functional. The second holds total variance fixed and shows that isotropic covariance minimizes the inverse-eigenvalue trace.

4.9. Matching a distribution through its projections

The nonlinear-risk argument gave a reason to prefer an isotropic Gaussian embedding distribution

under a stated family of downstream estimators and risk assumptions. It did not yet give a practical training objective. A neural encoder produces a high-dimensional batch of features; estimating and differentiating through its full probability density would be difficult. SIGReg avoids that density-estimation problem by comparing one-dimensional projections of the batch with one-dimensional Gaussian samples.

The construction joins three ideas. Characteristic functions identify probability distributions. The Cramér–Wold theorem says that all one-dimensional linear projections determine a multivariate distribution. The Epps–Pulley statistic provides a differentiable comparison between an empirical one-dimensional distribution and a Gaussian reference.

4.9.1. Why mean and covariance are not enough

Consider two scalar random variables. A standard normal variable has mean zero and variance one. A variable taking values -1 and $+1$, each with probability one half, has the same mean and variance. Their distributions are nevertheless different. The first is continuous; the second has two point masses.

Matching the first two moments constrains the centre and covariance of a distribution. It does not determine the full shape. The same limitation persists in a high-dimensional embedding. A zero mean and identity covariance do not imply that the batch is Gaussian, unimodal or even approximately Gaussian in every direction.

This matters because the Fisher-information argument did not identify only a covariance matrix. For fixed covariance, it identified the Gaussian as the distribution minimizing the score-based functional. A regularizer that controls only covariance does not enforce that target.

The question is how to compare a high-dimensional empirical distribution with a Gaussian reference without constructing a high-dimensional density estimator.

4.9.2. The characteristic function

For a scalar random variable U, define its characteristic function by

$$\varphi_U(t) = \mathbb{E}[e^{itU}], \qquad t \in \mathbb{R}.$$

The complex exponential has unit magnitude, so this expectation exists for every probability distribution. More importantly, the characteristic function uniquely determines the distribution. If two scalar random variables have the same characteristic function at every frequency t, they have the same law.

For a standard normal variable,

$$\varphi_0(t) = e^{-t^2/2}.$$

Given scalar samples $u_1, \ldots, u_N$, the empirical characteristic function is

$$\widehat{\varphi}_N(t) = \frac{1}{N} \sum_{n=1}^{N} e^{itu_n}.$$

It is a finite average of sine and cosine contributions. Unlike a high-order moment, it does not require raising samples to large powers. The bounded exponential also avoids the polynomial growth that can make high-order moment losses numerically unstable.

A characteristic function comparison still requires a range of frequencies and a weighting rule. Frequencies near zero describe broad distributional structure, while larger frequencies reveal finer differences. A numerical objective must decide how much weight each frequency receives and how the integral is approximated.

4.9.3. Why one-dimensional projections are sufficient in principle

Let $Z \in \mathbb{R}^D$ be a random embedding vector and let $a \in \mathbb{S}^{D-1}$ be a unit direction. Its scalar projection is

$$U_a = a^\top Z.$$

The Cramér–Wold theorem states that two probability distributions on $\mathbb{R}^D$ are equal if and only if their projections have the same one-dimensional distribution in every direction:

$$a^\top X \overset{d}{=} a^\top Y \quad \text{for all } a \in \mathbb{S}^{D-1} \quad \Longleftrightarrow \quad X \overset{d}{=} Y.$$

For a standard isotropic Gaussian $G \sim \mathcal{N}(0, I_D)$, every unit projection is standard normal:

$$a^\top G \sim \mathcal{N}(0, 1).$$

Consequently, a multivariate Gaussian-matching problem can be characterized through scalar normality in all directions. This is the mathematical reason projection-based distribution matching is possible.

The phrase *all directions* is essential. Equality on a finite collection of projections does not generally identify an arbitrary multivariate distribution. A finite set of directions may miss a discrepancy concentrated in another orientation. Random projections provide a stochastic sketch of the distribution, not a finite-sample proof of exact multivariate equality.

4.9.4. From a goodness-of-fit test to a training objective

For a unit direction a, project a batch of embeddings:

$$u_n = a^\top z_n, \qquad n = 1, \dots, N.$$

The Epps–Pulley statistic compares the empirical characteristic function of these projected samples with the standard-normal characteristic function:

$$T_{\mathrm{EP}}(u_{1:N}) = N \int_{-\infty}^{\infty} \left| \widehat{\varphi}_N(t) - e^{-t^2/2} \right|^2 w(t)\, dt.$$

Here $w(t)$ is a nonnegative frequency window that makes the discrepancy integrable and controls how different frequencies contribute. The target characteristic function and the window serve different purposes: the former describes the null distribution; the latter weights the comparison.

For a set of tested directions $\mathcal{A}$, the formal union-intersection testing construction uses the largest directional discrepancy:

$$T_{\max} = \max_{a \in \mathcal{A}} T_{\mathrm{EP}}(u^{(a)}_{1:N}).$$

A hypothesis test asks whether any tested direction gives evidence against the Gaussian null. The practical training regularizer uses a different aggregation:

$$\mathcal{L}_{\mathrm{SIGReg}} = \frac{1}{M} \sum_{m=1}^{M} T_{\mathrm{EP}} \left(a_m^\top z_1, \dots, a_m^\top z_N \right),$$

where $a_1, \dots, a_M$ are sampled unit directions. Averaging spreads gradient contributions across the sampled projections. The maximum and the average should not be conflated: they correspond to different statistical and optimization purposes. (Balestriero and LeCun, 2025)

The one-dimensional integral is evaluated numerically. In the paper's displayed algorithm, the frequency interval is discretized using 17 points over $[-5, 5]$, and a Gaussian weighting is used in the quadrature. The finite grid approximates the integral; it is not the definition of the underlying characteristic-function comparison. The paper also exploits symmetry of the integrand to increase the effective frequency coverage without doubling the computation.

4.9.5. Why resampling directions can work

A unit direction can be sampled by drawing a standard Gaussian vector and normalizing it:

$$a_m = \frac{g_m}{\|g_m\|_2}, \qquad g_m \sim \mathcal{N}(0, I_D).$$

Each minibatch then evaluates a finite set of random projections. Directions are resampled during training, so the same finite set need not be used throughout optimization. The number of directions seen across many updates can be substantially larger than the number used in one update.

This distinction explains why a small number of slices per minibatch does not mean that the model is constrained only along those directions forever. Across training, different projections expose different aspects of the embedding distribution. The method trades exact all-direction comparison for a stochastic approximation whose coverage accumulates over optimization.

The computation consists primarily of projecting the batch, evaluating scalar characteristic functions and accumulating quadrature values. Its cost depends on batch size N, feature dimension D, number of projections M, and number of frequency knots. The paper's linear-in-N complexity statement holds when the other quantities are fixed. It should not be read as independence from embedding dimension or number of projections.

For distributed training, each worker can accumulate local complex-exponential sums and combine them through an all-reduce operation. The statistic is therefore compatible with data-parallel training without gathering every projected sample on one device. The communication and projection counts remain part of the practical cost.

4.9.6. SIGReg inside LeJEPA

SIGReg specifies a distributional constraint. It does not say which views of an observation should agree. LeJEPA combines it with a direct multi-view agreement objective.

Let $z_{n,v}$ denote the embedding of sample n under view v. Suppose the first V_g views are global crops. Their per-sample mean is

$$\mu_n = \frac{1}{V_g} \sum_{v=1}^{V_g} z_{n,v}.$$

The agreement term draws every view embedding toward this global-view centre:

$$\mathcal{L}_{\text{pred}} = \frac{1}{NV} \sum_{n=1}^{N} \sum_{v=1}^{V} \|z_{n,v} - \mu_n\|_2^2.$$

SIGReg is applied to each view's batch distribution separately and then averaged across views:

$$\mathcal{L}_{\text{reg}} = \frac{1}{V} \sum_{v=1}^{V} \text{SIGReg}\left(\{z_{n,v}\}_{n=1}^{N}\right).$$

The combined objective is

$$\mathcal{L}_{\text{LeJEPA}} = (1-\alpha)\mathcal{L}_{\text{pred}} + \alpha\mathcal{L}_{\text{reg}}.$$

This formulation is different from the EMA target-encoder and predictor structure used by I-JEPA and original V-JEPA. The global-view mean provides the agreement target, while SIGReg constrains the distribution of each view. There is no requirement in the base objective for an EMA teacher or a separate predictor network. The regularizer also does not pool all views into one distribution before testing; each view distribution is regularized separately. This matters when global and local views have different crop distributions. (Balestriero and LeCun, 2025)

4.9.7. What SIGReg establishes, and what it approximates

The construction has a clear chain. The downstream-risk analysis motivates a Gaussian target under its assumptions. Cramér–Wold relates multivariate equality to equality of all one-dimensional projections. Characteristic functions identify each projected distribution. Epps–Pulley supplies a scalar goodness-of-fit statistic. Random directions, finite batches and numerical quadrature make the statistic trainable.

Each step has a boundary. A Gaussian target is not justified by Cramér–Wold; it is motivated by the earlier risk result. A finite number of projections is not an exact all-directions certificate. A finite batch gives a noisy empirical characteristic function. A finite frequency grid approximates the integral. A low training regularizer is therefore evidence that the sampled projections approximately match the Gaussian reference under the chosen numerical procedure, not proof that the population embedding distribution is exactly Gaussian.

SIGReg makes distributional shape part of the training objective rather than relying only on variance and covariance. It does so by sketching the distribution through random projections. The next section considers a different decomposition: centre the batch, constrain coordinate scale directly, and compare shape after scale has been removed.

Figure 4.14. The Cramér–Wold theorem links multivariate distributions to all one-dimensional projections.

Description: A high-dimensional embedding distribution is projected along many unit directions. Each projection is compared with a one-dimensional Gaussian reference. The theorem concerns all directions; training uses a finite random sample of them.

Figure 4.15. SIGReg compares empirical and Gaussian characteristic functions.

Description: The empirical characteristic function is evaluated over a finite frequency grid and compared with the standard-normal characteristic function. A weighted squared discrepancy is

integrated numerically, then averaged over sampled directions.

4.10. Separating centre, scale and shape

SIGReg compares projected characteristic functions with a Gaussian target. VISReg approaches distribution matching by separating three properties of a minibatch: its centre, the scale of each coordinate and the shape of the normalized distribution. The purpose is not to add three interchangeable anti-collapse penalties. Each term controls a different statistic, and the shape comparison is constructed so that coordinate scale is handled elsewhere.

4.10.1. The three properties of an embedding batch

Let the embeddings be rows of $Z \in \mathbb{R}^{N \times D}$. Define their mean and centred values by

$$\mu = \frac{1}{N} \sum_{n=1}^{N} z_n, \qquad \widehat{Z} = Z - \mu.$$

The mean determines where the cloud is centred. The coordinate standard deviations determine its scale along each feature axis. The normalized sample distribution determines its shape after those shifts and scales have been removed.

These properties can vary independently. A batch may be centred but have very small variance. It may have unit variance in every coordinate while exhibiting a non-Gaussian shape. It may match a reference shape after normalization but have the wrong unnormalized scale. A single covariance penalty does not impose all three conditions.

4.10.2. Directly constrain coordinate scale

Let $\sigma_j(\widehat{Z})$ denote the batch standard deviation of feature coordinate j. VISReg uses

$$\mathcal{L}_{\text{scale}} = \frac{1}{D} \sum_{j=1}^{D} \left(1 - \sigma_j(\widehat{Z})\right)^2.$$

The target standard deviation is one for each coordinate. The loss penalizes both contraction and expansion. Its derivative with respect to the standard-deviation statistic is

$$\frac{\partial \mathcal{L}_{\text{scale}}}{\partial \sigma_j} = -\frac{2}{D}(1 - \sigma_j).$$

At $\sigma_j = 0$, this scalar derivative is finite and nonzero. This is different from a loss whose

derivative with respect to variance itself vanishes at zero variance. The paper uses this comparison to motivate a direct scale objective.

The statement concerns the derivative with respect to σ_j, not a universal lower bound on the gradient of every encoder parameter. The chain rule still passes through the batch standard deviation and the encoder; its magnitude can depend on the data, representation, numerical stabilizer and network parameterization. The scale term supplies a direct correction at the statistic level, but it is not a theorem that every collapsed network can escape every initialization. (Wu et al., 2026)

4.10.3. Normalize scale before comparing shape

VISReg centres the batch and normalizes each coordinate by its standard deviation:

$$\tilde{Z} = \frac{\hat{Z}}{\mathrm{sg}(\sigma(\hat{Z})) + \epsilon},$$

where sg is stop-gradient and $\epsilon > 0$ stabilizes division. The normalization removes coordinate scale from the shape comparison. Stop-gradient means that the shape loss does not differentiate through the standard-deviation denominator.

This creates two distinct gradient paths. The scale term acts on the unnormalized centred batch and adjusts coordinate spread. The shape term compares normalized samples, treating the scale statistic as constant for differentiation. It is not an EMA target update: no slowly moving encoder is introduced. The stop-gradient is applied to the normalization statistic, not to a target network.

The centring term is

$$\mathcal{L}_{\mathrm{center}} = \|\mu\|_2^2.$$

It penalizes a shift of the embedding cloud away from the origin. It is separate from scale and shape because a distribution can have the desired coordinate variances and normalized shape while remaining offset.

4.10.4. Sliced Wasserstein matching

After centring and scale normalization, VISReg compares the projected empirical samples with projected samples from a standard Gaussian. For a unit direction w_k,

$$u_k = \tilde{Z} w_k \in \mathbb{R}^N.$$

Sort the projected values:

$$u_{k,(1)} \le \cdots \le u_{k,(N)}.$$

For the standard-normal reference, define equally spaced quantile levels and their Gaussian quantiles:

$$r_i = \frac{i}{N+1}, \qquad q_i = \Phi^{-1}(r_i), \qquad i = 1, \ldots, N.$$

In one dimension, the optimal quadratic transport coupling between two equally weighted empirical distributions matches their order statistics. The corresponding empirical squared Wasserstein distance is

$$W_2^2 = \frac{1}{N} \sum_{i=1}^{N} (u_{(i)} - q_i)^2.$$

Averaging this comparison over K_s random directions gives the sliced shape objective. The paper's displayed formula uses a squared Euclidean norm of the sorted vectors averaged over slices:

$$\mathcal{L}_{\text{shape}} = \frac{1}{K_s} \sum_{k=1}^{K_s} \left\| \operatorname{sort}(\tilde{Z} w_k) - q \right\|_2^2.$$

The paper's accompanying pseudocode computes a mean squared difference over the N samples and directions, which includes an additional factor $1/N$ relative to the printed squared-norm formula. This is a normalization discrepancy, not a different transport coupling. Since it changes the scale of the regularizer with batch size, a reproduction must follow one convention consistently and retune weights accordingly. The normalization equation also contains an ϵ stabilizer, while the printed algorithm excerpt divides by the detached standard deviation directly; the implementation choice should be recorded. (Wu et al., 2026)

The random projection turns a high-dimensional distribution comparison into several one-dimensional sorting operations. This is computationally attractive because one-dimensional optimal transport has a simple solution. It is still an approximation: a finite set of directions cannot certify exact equality of arbitrary multivariate distributions.

4.10.5. The complete VISReg objective

The regularization component combines the three separate constraints:

$$\mathcal{L}_{\text{reg}} = \lambda_s \mathcal{L}_{\text{scale}} + \lambda_h \mathcal{L}_{\text{shape}} + \lambda_c \mathcal{L}_{\text{center}}.$$

The method also has a view-agreement term. Let $z_{n,v}$ be the embedding for sample n under view v, and let the first V_g views be global crops. Their mean is

$$\mu_{g,n} = \frac{1}{V_g} \sum_{v=1}^{V_g} z_{n,v}.$$

The agreement term draws each view toward this per-sample global-view mean:

$$\mathcal{L}_{\text{pred}} = \frac{1}{NV} \sum_{n=1}^{N} \sum_{v=1}^{V} \|\mu_{g,n} - z_{n,v}\|_2^2.$$

The full objective is

$$\mathcal{L}_{\text{VISReg}} = (1 - \lambda)\mathcal{L}_{\text{pred}} + \lambda \mathcal{L}_{\text{reg}}.$$

There are four coefficients in this formulation: the outer trade-off λ, and the three weights inside the regularizer. The baseline sets the inner weights to one, with ablations that vary their relative contributions. (Wu et al., 2026)

VISReg and SIGReg therefore share a target motivation but use different mechanisms. SIGReg compares characteristic functions along projections. VISReg compares sorted projected quantiles after separately controlling centre and coordinate scale. The two objectives should not be treated as interchangeable or as the same regularizer with different names.

4.10.6. Computational cost and the role of slices

For K_s random directions, the projection cost is

$$O(NDK_s),$$

and sorting the N projected values for every direction costs

$$O(K_s N \log N).$$

The second term may be smaller than the projection cost at the dimensions considered, but both depend on batch size and slice count. More slices improve coverage of the distribution's directions, but increase computation.

The VISReg paper studies distributing slices across devices. Each worker can evaluate a subset of directions and aggregate the resulting loss. The authors report that more devices can compensate for fewer slices per device in their experiments. This is a scaling strategy observed under their configurations, not a theorem that a fixed number of per-device slices is sufficient at arbitrary dimension.

The experiments also compare VISReg on low-rank, long-tailed and degraded data, and evaluate downstream classification, transfer and dense prediction. The reported results support the method's decomposition in the tested settings. They do not prove that any finite projection set recovers the full distribution or that a Gaussian-shaped embedding is sufficient for an arbitrary physical task. (Wu et al., 2026)

4.10.7. Why the distinction matters

The difference between SIGReg and VISReg is instructive. A distributional target alone does not determine the best optimization interface. SIGReg works with characteristic functions. VISReg makes coordinate scale an explicit objective and removes it from the shape comparison with a stop-gradient normalization. Both are attempts to make distributional structure trainable without estimating a full density.

The chapter's central distinction remains unchanged. View agreement specifies which observations should share a representation. Distribution regularization constrains the geometry of the resulting features. Neither alone specifies what a world model must predict under a chosen action. That next requirement changes the conditioning variable, not merely the embedding regularizer.

Figure 4.16. VISReg assigns separate losses to centre, scale and normalized shape.

Description: The batch is first centred. A scale loss acts on the unnormalized standard deviations, while a shape loss compares sorted Gaussian projections of the normalized batch. A separate centre loss penalizes a nonzero mean.

Figure 4.17. Sliced Wasserstein matching reduces each projection to sorted samples.

Description: Empirical embeddings and Gaussian reference samples are projected along one direction, sorted, and matched by order statistic. Repeating this operation over random directions approximates a multivariate shape comparison.

4.11. When the target belongs to another modality

So far, the examples have used visual context and visual target features. The JEPA construction does not require the target to belong to the same modality as the input. VL-JEPA makes this point concrete: it predicts a text representation from visual evidence, with a language query specifying what should be expressed.

The important idea is not a catalogue of vision-language tasks. It is that the target space is a design choice. The visual encoder, query and target encoder play separate roles.

4.11.1. The query is not the answer

Let X_V denote an image or video, X_Q a text query and Y the target text. The visual encoder produces

$$S_V = f_\theta(X_V),$$

while a text encoder maps the answer to

$$S_Y = e_\psi(Y).$$

The predictor combines visual features and the query:

$$\widehat{S}_Y = p_\phi(S_V, X_Q).$$

The query tells the model which question is being asked; the target text is the answer against which its prediction is trained. For one video, a query might ask which object a person is about to touch. A different query could ask what action the person is performing. The visual evidence is shared, but the requested conditional prediction changes.

This is the same role played by a spatial query in I-JEPA or a spatiotemporal query in V-JEPA. The query specifies the requested target. It is not itself the target representation.

4.11.2. Why predict a text embedding?

A token-generative model predicts a sequence of discrete symbols. VL-JEPA instead estimates a continuous representation produced by a text encoder. Semantically related descriptions can be close in that target space even when their word sequences differ. The objective therefore compares meanings as represented by the chosen text encoder, rather than requiring the predictor to reproduce one exact sequence of tokens.

VL-JEPA uses a bidirectional InfoNCE objective between predicted and target text embeddings. For normalized predicted and target embeddings $\bar{\hat{s}}_i$ and $\bar{s}_i$, one direction has the form

$$\mathcal{L}_{\hat{S}\to S} = -\frac{1}{B}\sum_{i=1}^{B} \log \frac{\exp(\bar{\hat{s}}_i^\top \bar{s}_i/\tau)}{\sum_{j=1}^{B}\exp(\bar{\hat{s}}_i^\top \bar{s}_j/\tau)},$$

where B is batch size and τ is temperature. The reverse direction exchanges the predicted and

target roles. The positive pair is encouraged to align, while other batch targets provide contrastive comparisons. This is the paper's objective; it does not use SIGReg as its collapse regularizer. (Chen et al., 2026)

The choice of text encoder defines which linguistic distinctions remain visible. If two phrasings are close in its embedding space, the predictor is not strongly penalized for treating them as similar. If two answers differ in a small but important detail and the text encoder maps them too close, the target may be inadequate for that task. Embedding prediction does not remove the need to evaluate the target geometry.

4.11.3. Prediction and language generation are different operations

The text decoder is not required to train the embedding-prediction objective. Some tasks can use the predicted representation directly. For retrieval, it can be compared with candidate text embeddings. For classification, encoded label descriptions can be ranked against it. Captioning or open-ended question answering requires a readable string, so a decoder is invoked to turn the predicted embedding into language.

The published VL-JEPA system uses a frozen V-JEPA 2 ViT-L visual encoder. Its predictor is initialized from Transformer layers of Llama 3.2, while its text target encoder is initialized from EmbeddingGemma. Projection heads map predicted and target features into a shared 1,536-dimensional space. These are the paper's implementation choices, not requirements of cross-modal prediction. Training is staged: large-scale visual-text alignment is followed by query-conditioned supervised fine-tuning for tasks such as visual question answering. (Chen et al., 2026)

Because the embedding can be updated without decoding every frame into text, the paper also evaluates selective decoding for streaming video. In its EgoExo4D protocol, decoding at 0.35 Hz matches the reported performance of uniform decoding at 1 Hz, with approximately 2.85 times fewer decoding operations. This is a result for that evaluation protocol, not a general latency guarantee. (Chen et al., 2026)

4.11.4. A language query is not a control input

A query may ask a hypothetical question, but that wording does not make the model an intervention model. The predictor estimates a text embedding that answers the query. A controlled transition model estimates the consequence of a specified action:

$$\widehat{S}_Y = p_\phi(f_\theta(X_V), X_Q), \qquad \widehat{S}_{t+1} = F_\omega(S_t, A_t).$$

The first is cross-modal semantic prediction. The second is action-conditioned dynamics. To compare candidate robot commands, the predictor must receive the candidate action and be trained

on trajectories that connect actions to subsequent observations. A natural-language question about what might happen is not a substitute for that action interface.

VL-JEPA therefore extends the range of possible target modalities without collapsing the distinction between representation prediction and controlled dynamics. The target may be text, but the conditioning variable must still correspond to the quantity whose effect the model is meant to estimate.

Figure 4.18. Cross-modal prediction separates visual context, query and text target.

Description: Visual features and a text query enter a predictor. A separate text encoder supplies the target feature during training. The predicted embedding can be compared with candidate texts directly or decoded when a readable answer is required.

4.12. From passive video representations to robot prediction

The transition from video representation learning to physical prediction requires more than adding time to the input. A video encoder can learn features that describe motion and interaction, yet still lack an explicit variable for the action whose consequence a controller wants to estimate. V-JEPA 2 makes the separation concrete: it first learns a visual representation from large-scale passive data, then trains an action-conditioned predictor on robot trajectories. V-JEPA 2.1 strengthens the spatial density of the representation, while retaining a separate controlled predictor for its robot-planning experiments.

The important progression is therefore not a sequence of model names. It is a separation of responsibilities: the encoder supplies visual features; the controlled predictor estimates consequences under actions; the planner evaluates candidate action sequences.

4.12.1. Why passive video is a useful starting point

A large video corpus contains motion, object interaction, scene change and many visual regularities. An encoder pretrained on such data can provide features that are useful for recognizing objects and anticipating events. This gives a robot predictor a strong visual input without requiring the robot trajectories themselves to teach every visual concept from scratch.

But passive video is generated under the actions taken by the people or systems in the recordings. A model trained on these observations learns relations in the observed trajectory distribution. It does not automatically learn how the scene would change under a different action from the same state.

Let $X_{1:t}$ denote the observed history and Z_{t+1} a future feature. An action-free predictor estimates

$$p(Z_{t+1} \mid X_{1:t}).$$

A controlled predictor must estimate

$$p(Z_{t+1} \mid X_{1:t}, A_t),$$

where A_t specifies the action whose consequence is being considered. The second distribution can change when the action changes while history remains fixed. The first has no explicit input representing that alternative.

Passive pretraining and action-conditioned prediction therefore solve different problems. The pretrained encoder can make the visual input easier to interpret. It cannot supply a missing action variable by itself.

4.12.2. V-JEPA 2-AC adds the control interface

V-JEPA 2-AC is a post-training stage within the V-JEPA 2 paper. It freezes the pretrained visual encoder and learns a separate autoregressive predictor using robot video, action and end-effector state. The predictor receives an aligned history of visual features and control information, then estimates future visual features. This is the central separation in the V-JEPA 2-AC post-training stage (Assran et al., 2025).

The robot data come from DROID demonstrations. The paper uses end-effector position, orientation and gripper state to define a robot state vector. Actions are derived from changes in this state between adjacent observations. The data are therefore not merely passive images: the predictor receives a control signal aligned with the transition it must predict.

Let E be the frozen visual encoder, x_t an image, s_t the end-effector state and a_t the action. Write

$$z_t = E(x_t).$$

A causal predictor estimates the next feature from the history of visual features and controls:

$$\hat{z}_{t+1} = P_\phi\Big((z_i, s_i, a_i)_{i \le t}\Big).$$

The predictor uses block-causal attention: tokens at a time step can attend to current and previous information, but not future observations. This makes the temporal conditioning appropriate for an autoregressive transition model.

The paper combines teacher-forced prediction and a short rollout objective. Under teacher forcing, the predictor receives observed preceding features at each step and is trained to match the next encoded observation:

$$\mathcal{L}_{\mathrm{TF}} = \frac{1}{T} \sum_{t=1}^{T} \left\| P_\phi\Big((z_i, s_i, a_i)_{i \le t}\Big) - z_{t+1} \right\|_1.$$

At deployment, however, the model must consume its own previous predictions when rolling forward. Teacher forcing alone does not expose it to that input distribution. V-JEPA 2-AC therefore adds a two-step autoregressive rollout loss, in which a predicted feature is fed back to the predictor before the later prediction is compared with its encoded target:

$$\mathcal{L}_{\text{roll}} = \left\| P_\phi(a_{1:2}; s_1, z_1) - z_3 \right\|_1.$$

The complete post-training objective is the sum of these terms. The two-step rollout reduces the gap between training and inference, but it does not guarantee long-horizon accuracy. Errors may still accumulate when the predictor is applied repeatedly.

This design illustrates why the control interface is a modelling decision. The visual encoder can remain frozen while a predictor learns how observed robot actions relate to subsequent feature changes. The target now comes from a later observation, and the action is explicitly part of the conditioning information.

4.12.3. Planning uses a different objective from prediction

Once a predictor can estimate future features under candidate actions, a planner can compare action sequences. Let z_t be the current visual feature, s_t the current robot state and z_g the feature of a goal image. For a proposed action sequence $a_{t:t+H-1}$, the predictor estimates a future feature. A goal-conditioned energy can compare that estimate with z_g:

$$\mathcal{E}(a_{t:t+H-1}) = \left\| P_\phi(a_{t:t+H-1}; s_t, z_t) - z_g \right\|_1.$$

This energy is used to rank candidate actions during planning. It is not the same loss used to train the predictor. The training loss compares predicted features with observed future features. The planning energy compares an imagined outcome with a desired goal representation.

V-JEPA 2 uses the Cross-Entropy Method (CEM) to search over candidate action sequences. It repeatedly samples action sequences, evaluates them through the predictor, retains high-scoring candidates and updates the sampling distribution around them. The controller executes an initial action, obtains a new observation and plans again. This receding-horizon loop limits reliance on one long open-loop prediction.

The separation is essential. The predictor estimates consequences. The goal representation defines what outcome is sought. The planning algorithm searches over controls. A feature-prediction loss alone does not provide a goal, a cost or a search procedure.

4.12.4. What the robot experiments establish

The V-JEPA 2 paper evaluates the complete system on tabletop Franka Panda tasks, including

reaching, grasping and pick-and-place with image goals. The reported deployments include two laboratories and test whether the pretrained visual representation, robot-trajectory post-training and model-predictive controller transfer without task-specific robot training in the deployment environments.

The result is evidence about the combined system. It is not evidence that the action-free video encoder alone controls a robot. The controlled predictor is trained on DROID trajectories, and the planner uses an explicit goal and action search. Nor do the experiments establish general-purpose control across arbitrary robots, cameras or action spaces.

The limitations reveal why the interfaces matter. The action coordinates are defined relative to the robot base, while the image is obtained from a camera. If the relation between camera and robot coordinates is ambiguous, the predictor may have difficulty connecting visual features to actions. Long rollouts also accumulate prediction errors, while longer planning horizons increase the size and difficulty of the action search. Better visual features cannot remove every limitation in the control and planning interface.

4.12.5. V-JEPA 2.1 changes the density of visual supervision

V-JEPA 2.1 addresses a separate representation problem. In masked feature prediction, target tokens receive direct prediction loss, while visible context tokens primarily serve as inputs. A context token may therefore become useful for predicting distant masked targets without retaining sufficiently local information at its own position.

This matters for dense tasks such as depth estimation, segmentation and localization. A pooled representation can support scene recognition while discarding local spatial detail. Dense prediction requires the feature at a position to retain information about the corresponding part of the scene.

V-JEPA 2.1 adds a context loss on visible tokens. Let C denote visible positions and let $d_{\min}(i, M)$ be the distance, measured in spatiotemporal mask blocks, from visible token i to the nearest masked block. The loss assigns a larger weight to tokens nearer the masked region:

$$\lambda_i = \frac{\lambda}{\sqrt{d_{\min}(i, M)}}.$$

The weighting emphasizes the visible features closest to the boundary between context and target. It encourages local consistency across that boundary without applying the same target constraint uniformly to every visible token. The full objective retains the masked-token prediction loss and adds the visible-context loss.

The method also applies deep self-supervision at three intermediate encoder blocks and the final block. The dense context objective and its weighting follow the V-JEPA 2.1 paper (Mur-Labadia et

al., 2026). Intermediate features are combined and supervised alongside the final representation. The motivation is that local spatial information should not be required to survive only in the last encoder layer after many rounds of global feature mixing.

These changes affect where and which visual features are supervised. They do not add an action variable. V-JEPA 2.1 is still a video representation-learning method; its robot-planning experiments use a separately trained action-conditioned predictor.

4.12.6. Dense features can improve a controller without becoming a controller

The V-JEPA 2.1 robot experiments retain the action-conditioned predictor and planning interface used in V-JEPA 2-AC, while replacing the visual encoder with the denser representation. Under the matched configuration reported for cup grasping, the success rate rises from 60% with V-JEPA 2 features to 70% with V-JEPA 2.1 features. This is a ten-percentage-point difference under that particular comparison.

The paper also reports a higher V-JEPA 2.1 success rate under a different planning configuration with more CEM iterations, a longer horizon and a larger time budget. That result should not be presented as a matched comparison with the 60% or 70% rows. The search budget and horizon are part of the experiment.

The limited conclusion is that more spatially informative visual features can improve the information available to a separately trained action-conditioned predictor and planner. It does not show that dense masked prediction alone creates a physical transition model.

4.12.7. Three questions that should remain separate

The progression from V-JEPA 2 to V-JEPA 2.1 makes three design questions visible:

1. **What should the representation preserve?** The pretraining target, mask geometry, visible-token supervision, intermediate losses, data and tokenizer determine which visual distinctions are rewarded.
2. **How should the model predict under a chosen action?** A controlled predictor needs aligned observations, actions and relevant robot state. Its data must cover the state-action combinations on which it will be used.
3. **How should actions be selected?** A planner needs a goal or cost, an action parameterization, a search strategy, constraints and feedback.

A change to the encoder addresses the first question. Adding an action-conditioned predictor addresses the second. A planner addresses the third. Evidence at one level does not automatically establish competence at the others.

Figure 4.19. V-JEPA 2 separates passive visual pretraining from controlled post-training.

Description: A visual encoder is pretrained on passive images and videos. It is then frozen while a separate predictor learns from robot observations, end-effector state and aligned actions. A planner evaluates candidate controls using the resulting predictor and a goal representation.

Figure 4.20. V-JEPA 2.1 adds supervision to visible tokens without adding actions.

Description: The original masked prediction loss supervises withheld target tokens. V-JEPA 2.1 adds distance-weighted supervision to visible context tokens and applies losses at intermediate encoder depths. The action-conditioned predictor remains a separate component for robot planning.

4.13. Learning what an action does

A model trained on video learns relationships between observations. A model used for control must answer a different question: what would happen if the agent chose a particular action now? The distinction is between predicting an outcome under the behaviour recorded in a dataset and predicting the outcome under a specified intervention.

An action input is necessary for the second problem, but not sufficient. The data must contain information that identifies how outcomes vary with actions, and the representation must preserve the state variables that affect those outcomes.

4.13.1. Observed actions and chosen actions define different distributions

Let H_t denote the observation history available at time t, A_t the action taken, and Y_{t+1} a future observation or representation. A passive dataset provides samples from a joint distribution such as

$$p(H_t, A_t, Y_{t+1}).$$

From it, a model can estimate the conditional distribution

$$p(Y_{t+1} \mid H_t, A_t = a).$$

This is a statement about examples in which the behaviour policy happened to choose a under histories like H_t. A controller, however, wants the distribution that would result from setting the action to a:

$$p(Y_{t+1} \mid H_t, \mathrm{do}(A_t = a)).$$

The intervention notation distinguishes fixing an action from selecting observed cases where that action occurred. The two distributions agree only under additional assumptions. If the action is

selected using information about the state that also affects the future, conditioning on the action can select a biased subset of histories. A behaviour policy may choose to push an object only when it is already close to the gripper, for example. Observed pushes are then correlated with the geometry that made the push possible.

4.13.2. What must be identified

A common way to express the controlled problem is through a state S_t that summarizes the history:

$$S_t = f_\theta(H_t).$$

For this state to support controlled prediction, it should preserve the future distribution relevant to the action space:

$$p(Y_{t+1} \mid H_t, A_t) = p(Y_{t+1} \mid S_t, A_t).$$

This is a task-relative sufficiency requirement. If hidden velocity, contact state or gripper configuration affects the outcome of an action but is absent from S_t, a predictor may give different answers for histories that it has represented identically.

Even a sufficient state does not automatically identify interventions from arbitrary observational data. At least three conditions must be considered.

Consistency. The recorded outcome under an observed action must correspond to the potential outcome associated with that action. The action representation must have a well-defined meaning and be aligned with the transition interval.

Action support. The dataset must contain relevant actions in the states where the model will be queried. If the behaviour policy never tries to move left from a particular state, the data provide no direct evidence about that transition. This is often called positivity or overlap: the actions to be evaluated must have nonzero probability under the data-collection policy in the relevant states.

Conditional exchangeability. After conditioning on the state information supplied to the predictor, action selection should contain no unobserved information that also changes the outcome. In a controlled simulator, one may know the complete state and intervention mechanism. In real robot data, unobserved contact, object motion or policy information can violate this assumption.

These conditions explain why recording actions beside video is not enough. Alignment supplies the input variable. Coverage and state sufficiency determine whether the resulting conditional model can support the counterfactual questions a planner will ask.

4.13.3. A controlled feature predictor

Let a visual encoder map the current observation to features $Z_t = E(X_t)$. Let A_t be the control command and R_t any additional robot state needed to describe the actuator or end effector. A one-step transition predictor can be written as

$$\widehat{Z}_{t+1} = F_\phi(Z_{\le t}, A_{\le t}, R_{\le t}).$$

For a Markov-style model, the history may be replaced by a learned state S_t:

$$\widehat{S}_{t+1} = F_\phi(S_t, A_t).$$

Training on aligned trajectories compares predictions with the encoded observations that actually followed the recorded actions. For a feature loss ℓ,

$$\mathcal{L}_{\text{transition}} = \mathbb{E}\left[\ell(F_\phi(S_t, A_t), \operatorname{sg}(S_{t+1}))\right].$$

The target may be a visual feature, a structured state vector or a combination. The stop-gradient indicates that the target representation is not updated by this transition loss; the precise target update depends on the chosen architecture. A predictor trained with teacher forcing receives the observed previous state at each step. If it will be rolled forward at inference, training should also examine the errors produced when predicted states become subsequent inputs.

The action-conditioned post-training of V-JEPA 2 is one concrete example of aligning robot end-effector changes with subsequent visual features (Assran et al., 2025). The action representation must match the control system. A discrete action label, a joint-position command, a velocity target and a Cartesian end-effector displacement describe different control interfaces. The time interval over which the action applies must also be explicit. A command observed at 10 Hz cannot be treated as though it were a continuous force applied over an unspecified interval.

4.13.4. Data collection determines which questions can be answered

Suppose a robot dataset consists mostly of successful grasps. The predictor may learn the visual transition that usually accompanies a grasp. It may not have enough evidence to predict what happens if the gripper closes too early, moves sideways or opens during contact. The model can interpolate between observed actions only to the extent that the data and architecture support that interpolation.

Coverage can be improved by collecting varied trajectories, randomizing safe control parameters or using a simulator to generate interventions. In real systems, exploration is constrained by collision risk, actuator limits and the possibility of damaging objects. The dataset must therefore be

designed around the action space that the planner will search, not merely around the actions that were convenient to record.

A useful split holds out transitions by state and action, rather than randomly splitting adjacent video frames from the same trajectory. Random frame-level splits can place nearly identical contexts in both training and test sets, overstating generalization. Evaluation should ask whether predictions remain accurate for unseen state-action combinations that lie within the intended deployment support.

4.13.5. Testing the intervention claim

A transition predictor should be evaluated on questions that distinguish observation from intervention. In a simulator, one can hold the initial state fixed, apply several candidate actions and compare predicted outcomes with the corresponding simulated transitions. This tests whether changing the action changes the prediction in the correct way.

Other useful tests include one-step prediction error on held-out trajectories; error as a function of rollout horizon when predictions are fed back; performance on actions that are rare but still within the training support; sensitivity to changes in camera, object geometry or actuator calibration; and closed-loop success when a planner uses the predictor to select actions.

No single metric establishes causal correctness for every environment. A good one-step feature match can coexist with poor counterfactual prediction. A predictor can match the average outcome under a behaviour policy while failing under an action distribution chosen by an optimizer.

The essential requirement is that the model be tested on the conditional questions it will be asked at deployment. If the planner will compare several actions from the same state, evaluation must compare those alternatives, not only replay observed trajectories.

Figure 4.21. Conditioning on an observed action is not automatically an intervention.

Description: A behaviour policy selects actions using the current state and produces observed outcomes. An intervention fixes a candidate action while holding the starting state fixed. Identifying the intervened outcome requires aligned actions, adequate action support and sufficient control of confounding variables.

4.14. Planning with a learned predictor

A transition model becomes useful for control when it can evaluate the consequences of candidate actions. The planner supplies the objective that distinguishes desirable futures from undesirable ones. Prediction and planning are linked through imagined trajectories, but they optimize different quantities.

4.14.1. From a one-step transition to a trajectory

Suppose the model receives a state S_t and an action A_t, and predicts the next state:

$$\widehat{S}_{t+1} = F_\phi(S_t, A_t).$$

Applying the predictor repeatedly gives an imagined trajectory under an action sequence:

$$\widehat{S}_{t+k+1} = F_\phi(\widehat{S}_{t+k}, A_{t+k}).$$

The model can therefore evaluate a sequence before the agent executes it. For a finite horizon H, a general planning objective is

$$\min_{A_{t:t+H-1}} \mathbb{E}\left[\sum_{k=0}^{H-1} c(\widehat{S}_{t+k}, A_{t+k}) + V_{\text{term}}(\widehat{S}_{t+H})\right].$$

The immediate cost c evaluates states and actions during the imagined trajectory. The terminal value V_{term} estimates the value beyond the explicit rollout. Without a terminal term, a finite-horizon planner can prefer a trajectory that makes quick progress during the search window but ends in a poor state just beyond it.

The objective is not the same as the feature-prediction loss. The predictor is trained to match observed transitions. The planner minimizes a task cost evaluated on predicted states. A low transition error does not imply that the task cost is low, and a low task cost under the model does not guarantee that the real environment will produce the imagined state.

4.14.2. Image goals and feature-space distance

In image-goal control, the goal may be an image X_g rather than a vector of physical coordinates. An encoder maps the current observation and goal image into features:

$$Z_t = E(X_t), \qquad Z_g = E(X_g).$$

A simple terminal cost is a feature discrepancy,

$$c_{\text{goal}}(\widehat{Z}_{t+H}, Z_g) = \|\widehat{Z}_{t+H} - Z_g\|_1.$$

This cost is useful only to the extent that the representation preserves distinctions relevant to the goal. If two visually different states are mapped to similar features, the planner may consider them interchangeable. If a feature distance ignores contact, collision or object orientation, it may reward

an unsafe or incomplete outcome.

Goal matching is not a safety specification. Collision constraints, actuator limits and force limits must be represented separately when they matter. A latent distance can rank visual similarity; it does not certify that the path between two feature vectors is physically safe.

4.14.3. Searching over action sequences

The planner must solve an optimization problem over a sequence of actions. Gradient-based optimization is possible when the model and action parameterization are differentiable, but it can be sensitive to non-smooth constraints and model errors. Sampling-based methods search by evaluating candidate sequences.

The Cross-Entropy Method (CEM), used in V-JEPA 2, maintains a distribution over action sequences. It samples candidates, evaluates their predicted costs, selects an elite subset with the best scores, and updates the sampling distribution toward those candidates. Repeating this procedure concentrates samples in regions of action space that the model predicts to be promising. V-JEPA 2 uses CEM to optimize image-goal action sequences in its robot-planning system (Assran et al., 2025).

The search budget matters. More samples and iterations can find lower-cost trajectories under the model, but the model may be inaccurate for the states and actions that the optimizer discovers. A stronger optimizer can therefore make real-world performance worse if it is allowed to exploit errors in the learned transition model.

The planner is optimizing the model’s prediction, not the true environment. This is the source of model exploitation.

4.14.4. Receding-horizon feedback

An open-loop plan would execute the entire optimized action sequence without observing the environment again. Small model errors can then accumulate over the full horizon. A common alternative is receding-horizon control:

1. Observe the current environment and encode its state.
2. Optimize a candidate action sequence over a finite horizon.
3. Execute only the first action or short initial segment.
4. Receive a new observation and update the current state.
5. Replan from the new observation.

The next observation corrects some of the prediction error before the remaining actions are executed. This feedback loop does not make the model exact, and it does not guarantee safety. It

limits how long the controller relies on one open-loop imagined trajectory.

The horizon remains a trade-off. A short horizon limits accumulated prediction error but may miss actions whose benefit appears later. A long horizon can represent delayed consequences, but the predictor must remain accurate through repeated application and the search becomes more demanding.

4.14.5. Why optimization can make model error worse

Let the true transition be $F^\star$ and the learned model be

$$F_\phi(s,a) = F^\star(s,a) + \epsilon_\phi(s,a),$$

where ϵ_ϕ denotes prediction error in an appropriate state representation. If the planner searches for actions that minimize model-predicted cost, it may find regions where ϵ_ϕ makes the predicted outcome look unusually good.

This is not random evaluation error. The optimizer actively selects actions based on the model's outputs. Even if prediction errors are small on average under the training distribution, the minimum predicted cost across many candidates can be biased toward sequences with optimistic errors.

The effect intensifies with search. More samples increase the chance of finding a sequence that exploits a model defect. The optimizer does exactly what it is designed to do: it finds the best candidate according to the supplied model. The failure is that the model is not accurate enough over the region the optimizer explores.

Mitigations include collecting data near planner-proposed states, penalizing uncertain predictions, using ensembles to identify disagreement, constraining candidate actions to supported regions, and replanning frequently. Each helps only under assumptions. Ensemble agreement does not prove correctness if all models share the same bias. A penalty on uncertainty is useful only if the uncertainty estimate is calibrated enough to detect unfamiliar states. Restricting actions to the data support can reduce extrapolation but may also exclude useful behaviours.

4.14.6. Multiple futures and uncertainty

For a fixed state and action, the next state may be stochastic because of unobserved variables, sensor noise or genuinely random dynamics. A deterministic predictor trained with a pointwise loss estimates a statistic of the conditional target, such as a mean under squared error or a median under absolute error. It may not preserve several distinct outcomes.

For planning, uncertainty matters when different plausible futures have different costs. One option is to predict a distribution or several latent hypotheses and evaluate candidate actions across those

outcomes. Another is to maintain an ensemble of transition models and examine their disagreement. These are different approximations to uncertainty; neither guarantees that all relevant futures have been represented.

A model that outputs one plausible feature sequence can still be useful in near-deterministic settings. In ambiguous settings, averaging incompatible futures can produce a feature vector that corresponds to no realizable state. The planner must then be designed around the output semantics of the predictor rather than treating every predicted vector as a physical state.

4.14.7. Prediction quality, task value and safety

Three evaluations should remain separate.

Transition accuracy measures whether the model predicts observed or intervened outcomes.

Task value measures whether the imagined outcome satisfies the goal or cost function.

Safety measures whether the trajectory respects constraints on collision, force, workspace, velocity or other hazards.

A model can be accurate while the planner chooses an undesirable action because its cost is wrong. A model can predict a goal-like final image while the intermediate trajectory collides with an obstacle. A trajectory can satisfy a learned feature cost while violating an unmodelled physical constraint.

This is why a world model does not replace a task specification or a safety layer. It supplies imagined consequences that other components use. Closed-loop deployment must evaluate the full system under the hazards and constraints of the actual environment.

Figure 4.22. Receding-horizon planning executes an action and then replans.

Description: A planner samples and evaluates action sequences using a learned transition model. It executes only the first action, obtains a new observation and repeats the optimization from the updated state.

Figure 4.23. Search can exploit optimistic prediction errors.

Description: Several candidate trajectories have similar true outcomes, but one is assigned an artificially low model-predicted cost. An optimizer selects that trajectory because it appears best under the model, even though it fails when executed in the environment.

Figure 4.24. A point prediction can hide distinct plausible futures.

Description: One state-action pair leads to several separated outcome modes. A deterministic point estimate may lie between them, whereas a distribution or multiple hypotheses can preserve the

alternatives relevant to planning.

4.15. The agent around the world model

A transition predictor does not define a complete agent. The agent also needs a way to perceive the environment, choose objectives, select actions, retain relevant history and decide when to plan. LeCun's proposal for autonomous machine intelligence is useful because it makes these responsibilities explicit. It should be read as a research architecture and position paper, not as evidence that the complete system has been validated end to end. (LeCun, 2022)

4.15.1. Six responsibilities, not six interchangeable networks

The proposal describes six top-level modules:

Perception maps sensor observations into a representation of the current situation. Its output should expose the distinctions needed by prediction and control, rather than merely compressing the raw input.

World Model predicts how the represented situation may evolve. It may represent multiple possible futures and should be able to condition those futures on actions or other variables that affect the transition.

Cost evaluates predicted states and trajectories according to goals, preferences and constraints. It supplies the criterion by which imagined futures are compared.

Actor selects actions. It can implement a direct policy or participate in a deliberative planning process.

Short-Term Memory retrieves information from previous observations when the current input is insufficient to determine the relevant state.

Configurator specifies the task, subgoals and operating mode of the agent, and configures the other modules for the current objective.

The physical environment remains outside this internal module list. It generates observations and receives actions. This distinction prevents an architecture diagram from suggesting that the World Model directly changes the world.

The names do not imply that each module must be a separate neural network. A practical implementation may share parameters or combine components. The purpose of the decomposition is to separate responsibilities that have different objectives and failure modes.

4.15.2. Prediction and evaluation must remain separate

The World Model should answer what may happen under a proposed action. Cost should answer whether that predicted outcome is desirable. The distinction can be written as

$$\widehat{S}_{t+1} = F_\phi(S_t, A_t), \qquad J(A_{t:t+H-1}) = \sum_{k=0}^{H-1} c(\widehat{S}_{t+k}, A_{t+k}) + V_{\text{term}}(\widehat{S}_{t+H}).$$

The transition model F_ϕ describes consequences. The cost c evaluates them. The same transition model can be reused for different tasks by changing the objective, provided the representation and action interface retain the information those tasks require.

LeCun places Intrinsic Cost and a Trainable Critic within the Cost module. Intrinsic Cost represents built-in preferences or constraints; the Trainable Critic estimates task-related cost from experience. They are subcomponents of Cost, not additional top-level modules. This separation allows some preferences to be specified independently of a learned task value.

The decomposition also exposes two distinct failure cases. If the model predicts the wrong future, a correct cost cannot repair the transition. If the model predicts accurately but the cost function rewards the wrong outcome, the planner can select an undesirable action. The two modules require separate evaluation.

4.15.3. Mode 1: direct reactive behaviour

Mode 1 maps a current representation directly to an action:

$$A_t = \pi_\psi(S_t).$$

It does not need to search over imagined futures at every decision. A trained reactive policy can act quickly and with relatively low computational cost. This is useful when the control loop must run at high frequency or when planning is too expensive for every step.

The limitation is that the policy must have learned an appropriate response for the states and tasks it encounters. It does not explicitly compare a new set of candidate trajectories when the goal changes. Its behaviour outside the training distribution depends on generalization rather than on an online search over predicted consequences.

4.15.4. Mode 2: deliberation through imagined futures

Mode 2 uses the World Model and Cost to evaluate candidate action sequences. It searches for actions whose predicted consequences have low cost. The planner can adapt to a new goal or a changed objective without retraining the reactive policy.

This flexibility has a computational price. Search requires repeated model evaluations, and the chosen action depends on the accuracy of the imagined trajectories. The planner may also exploit prediction error, as described in the preceding section. Mode 2 is not automatically more reliable than Mode 1; it is more explicit about the candidate futures it evaluates.

The two modes therefore solve different operating problems. Mode 1 is inexpensive at execution but may be less flexible. Mode 2 can deliberate over alternative futures but consumes more computation and depends on the model being reliable in the regions explored by the search.

4.15.5. Distilling deliberation into a reactive policy

A planner can generate training targets for a faster policy. Suppose Mode 2 chooses an action $A^{\mathrm{M2}}(S)$ after searching from state S. A reactive policy can be trained to imitate those decisions:

$$\mathcal{L}_{\mathrm{distill}} = \mathbb{E}_{S \sim \mathcal{D}_{\mathrm{M2}}} \left[\left\| \pi_\psi(S) - A^{\mathrm{M2}}(S) \right\|_2^2 \right].$$

The resulting Mode 1 policy approximates the planner over the states represented in the distillation data. It can reduce the computational cost of repeatedly solving the planning problem.

Distillation does not transfer a guarantee of correctness. If Mode 2 exploits model errors, the selected actions can be wrong, and the reactive policy can learn to reproduce them. If the deployment state distribution differs from the distillation distribution, Mode 1 may not reproduce the planner's behaviour reliably. The improvement is computational: decisions obtained through search are converted into a faster mapping. It is not a proof that the policy is optimal outside the sampled states.

4.15.6. Short-Term Memory as associative retrieval

The current observation may not contain enough information to infer the relevant state. An object can be temporarily occluded; an earlier action can determine whether a container is open; a task may require remembering which subgoal has already been completed.

The Short-Term Memory in LeCun's proposal is associative rather than merely chronological. A key derived from the current context is used to retrieve stored content associated with similar past situations. Abstractly,

$$M_t = \mathrm{Retrieve}(K_t, \mathcal{M}),$$

where K_t is the current retrieval key and $\mathcal{M}$ is the memory. Retrieved content can be combined with the current representation before prediction or planning.

Associative retrieval is useful when the present observation is ambiguous but a past episode

contains relevant information. It also introduces a failure mode: the key must retrieve memories based on task-relevant similarity. If the representation ignores the distinction that determines which memory is relevant, the system may retrieve a plausible but incorrect episode. Memory therefore inherits the invariances and omissions of the perception system.

The architecture leaves important implementation questions open. What should be stored? How should keys and values be formed? When should a memory be updated or forgotten? How should the agent distinguish a genuinely similar situation from one that only looks similar in the current representation? These are part of the memory problem, not details solved by assigning a module name.

4.15.7. Hierarchical prediction separates time scales

A single predictor operating at the frame or control-step level may be inefficient for tasks with long horizons. A hierarchy can predict at several temporal scales. A high-level predictor proposes an abstract subgoal or a compressed future segment; a lower-level predictor estimates the intermediate states and actions required to realize it.

Let $s_t^{(0)}$ denote a low-level state representation. A higher-level representation can summarize a longer interval:

$$s_k^{(1)} = G_1(s_{t:t+\Delta}).$$

A high-level predictor can then estimate a temporally extended target:

$$\widehat{s}_{k+1}^{(1)} = F_1(s_k^{(1)}, a_k^{(1)}).$$

A lower-level predictor refines this target into shorter-horizon states and actions. The abstraction reduces the need for one model to predict every low-level detail over the entire horizon.

The hierarchy also creates a consistency requirement. The high-level prediction must be realizable by the lower-level controller. If the high-level representation discards a constraint needed for execution, the lower-level model may be unable to realize the proposed subgoal. Hierarchical JEPA is therefore not simply a stack of identical encoders. Each level needs a target and temporal abstraction appropriate to its role, together with a relationship between levels that preserves executability.

4.15.8. The Configurator is an open problem

The Configurator is the least specified of the six modules. In the proposal, it configures the task, goals, constraints and relation between reactive and deliberative operation. It is intended to

determine what the agent should work on and how the other modules should be arranged for that task.

That role raises questions which are not solved by drawing a box. Where do goals come from? How are constraints represented? How does the agent know when the task has changed? How are conflicts between intrinsic preferences and externally specified objectives resolved? What prevents the Configurator from selecting a convenient but inappropriate objective?

LeCun's proposal makes this responsibility explicit, which is useful for reasoning about a general agent. It does not supply a complete general algorithm for the Configurator or demonstrate that the six modules form a reliable autonomous system.

4.15.9. What a JEPA contributes

A JEPA contributes a representation and a predictive relation. In an action-free visual model, it can supply features that summarize a scene and support prediction of withheld or future representations. In an action-conditioned system, a separate predictor can estimate consequences under proposed controls. The agent still requires a cost, a policy or planner, feedback, memory and a mechanism for configuring the task.

The architectural conclusion is that representation prediction is one responsibility inside a larger agent. Its value depends on whether the representation preserves the distinctions needed by the predictor and whether the predictor remains reliable over the actions considered by the planner.

Figure 4.25. The proposed agent separates six responsibilities.

Description: Perception supplies representations to the World Model. The World Model predicts imagined states, which are evaluated by Cost. Actor selects actions, Short-Term Memory retrieves relevant past information, and Configurator specifies tasks and modes. Intrinsic Cost and Trainable Critic are shown inside Cost, not as additional top-level modules.

Figure 4.26. Mode 1 executes a reactive policy; Mode 2 searches imagined futures.

Description: Mode 1 maps a representation directly to an action. Mode 2 uses the World Model and Cost to compare action sequences. Planner-selected actions can train a cheaper Mode 1 policy, but distillation transfers the planner's behaviour only over the training distribution.

Figure 4.27. Hierarchical prediction separates temporal scales.

Description: A high-level predictor proposes an abstract subgoal or temporally extended state. A lower-level predictor refines it into intermediate states and actions. The hierarchy reduces the need for one predictor to represent every detail at every time scale.

4.16. Following one JEPA training step

The architecture becomes easier to understand when followed through one batch. Consider an I-JEPA training step with a batch of images. Each image is divided into patches, a context mask selects visible positions, and target masks select regions whose features must be predicted. The context encoder, target encoder and predictor do not receive identical inputs. The order of operations determines what information can flow through the loss.

4.16.1. From pixels to patch tokens

Let the image batch be

$$X \in \mathbb{R}^{B\times 3\times H\times W},$$

where B is batch size. A patch embedding layer maps each image to L tokens of dimension D_e:

$$X \longrightarrow P \in \mathbb{R}^{B\times L\times D_e}.$$

Positional embeddings are added so the encoder can distinguish patch content from patch location. The resulting sequence is the input to the transformer encoder.

The exact number of tokens depends on image size and patch size. For a square image divided into $p \times p$ patches, $L = (H/p)(W/p)$. This notation describes the conceptual tensor; implementations may flatten, transpose or group dimensions differently.

4.16.2. Construct context and target positions

The mask sampler produces context positions C and one or more target blocks $T_1, \ldots, T_M$. Conceptually, the context indices have shape

$$C \in \mathbb{N}^{B\times L_C},$$

and a target block has indices

$$T_m \in \mathbb{N}^{B\times L_T}.$$

The indices select tokens from the patch sequence. They do not themselves contain image information. Their job is to specify which tokens are visible and which target locations will be queried.

The online context encoder receives only the visible tokens:

$$Z_C = f_\theta(P_C) \in \mathbb{R}^{B\times L_C\times D_e}.$$

The target encoder receives the full patch sequence:

$$H = f_{\bar{\theta}}(P) \in \mathbb{R}^{B \times L \times D_e}.$$

The selected target features are

$$H_{T_m} = [H]_{T_m} \in \mathbb{R}^{B \times L_T \times D_e}.$$

This difference is important. The context encoder cannot read target tokens, while the target encoder processes the full image before features are selected. Target features can therefore incorporate surrounding image context from the target branch.

4.16.3. The target branch and gradient path

The target encoder is evaluated without gradient tracking in the prediction step. Its output is detached before entering the loss. The prediction loss therefore updates the online context encoder and predictor, but not the target encoder.

The target parameters change after the online optimizer step through an exponential moving average:

$$\bar{\theta}_{k+1} = m_k \bar{\theta}_k + (1 - m_k)\theta_{k+1}.$$

The target update is parameter-level. It is not a gradient from the target feature tensor. Keeping these operations separate avoids the misleading statement that the target encoder is trained by the same loss but with a smaller learning rate. In this recipe, the target branch follows the online encoder through its EMA rule.

4.16.4. The target position is a query

The predictor needs both the context features and the requested target locations. The target mask is converted into positional query embeddings. The predictor combines these queries with context features and produces one predicted feature per target position:

$$\hat{H}_{T_m} = g_\phi(Z_C, Q_{T_m}) \in \mathbb{R}^{B \times L_T \times D_e}.$$

The query carries the location, not the answer. Supplying the target feature itself would defeat the prediction task.

The predictor may project context tokens and query embeddings into a different internal dimension. Its output is then projected back into the target feature dimension so it can be compared

with H_{T_m}. The tensor shape is a statement about the interface; internal attention blocks, normalization and projection layers determine how the mapping is computed.

4.16.5. Loss, optimizer and EMA order

The implementation computes a feature loss for each target block and averages across target positions and the batch. In the released I-JEPA training code, the target features are normalized and detached, and the prediction loss is implemented with smooth-L_1 regression. The paper's concise mathematical formulation is written as an L_2-style feature distance. A reproduction should retain this distinction rather than treating equation and implementation as exactly identical. The target-feature selection, detach path and optimizer/EMA ordering are checked against the released I-JEPA training code (Assran et al., 2023; official implementation).

A conceptual training step is:

1. Sample an image batch and context/target masks.
2. Embed patches and add positional information.
3. Run the context encoder on visible tokens.
4. Run the target encoder on the full image without gradients.
5. Select target features and build position queries.
6. Predict target features from context and queries.
7. Compute the feature discrepancy against detached targets.
8. Backpropagate through predictor and context encoder.
9. Update their parameters with the optimizer.
10. Update target parameters by EMA.

The order matters. The target features used by the current loss are produced before the EMA update. The target update then prepares a slowly changing target for a later step.

4.16.6. What changes for video

In V-JEPA, the image token index is replaced by a spatiotemporal token index (t, h, w). Patch embedding becomes tubelet embedding. The mask sampler selects spatiotemporal blocks, and target queries include temporal positions as well as spatial positions. The target encoder processes the full clip; the context encoder processes the visible tokens. The same conceptual loss path remains: predict target features from context and queries.

The temporal target sampler does not automatically impose a causal boundary. Whether future frames are visible to the context encoder depends on the mask design. For an action-conditioned

predictor, the inputs change more substantially: actions and relevant robot state enter the transition predictor, which is trained on aligned trajectories. That is not simply V-JEPA with a different mask.

4.16.7. What changes in predictor-free view agreement

LeJEPA illustrates a different use of the representation objective. Rather than predicting masked target tokens through an EMA target branch and a separate predictor, its base objective draws embeddings from multiple views toward the per-sample mean of global-view embeddings and regularizes the distribution of each view with SIGReg.

The computational graph and statistical objective are therefore different. It is reasonable to discuss both under the wider family of joint-embedding self-supervised methods, but the term predictor should be reserved for a network that actually estimates a target representation from a specified context and query. A direct view-agreement term does not have the same information flow as masked feature prediction.

4.16.8. What to inspect in an implementation

When reading code for a JEPA training step, trace the tensors rather than relying on module names. Identify which image or clip is passed to each encoder, where masks are applied, when target features are selected, how positional queries are built, which tensors are detached, and exactly which parameters the optimizer updates. Then locate the EMA update and verify when it occurs relative to the optimizer step.

This procedure can expose errors that a high-level block diagram hides. Applying target masks before the target encoder when the method requires full-image contextualized targets changes the task. Passing target features to the predictor leaks the answer. Updating target parameters through the loss instead of the intended EMA rule changes the gradient path. Treating a full-clip mask as causal changes the interpretation of the video objective.

A training implementation is therefore part of the scientific definition of the method. The paper explains the objective; the code specifies operational details that may be necessary for an exact reproduction. Neither should be substituted silently for the other.

Figure 4.28. One I-JEPA step from patches to target update.

Description: A batch is converted into patch tokens. The context encoder receives visible tokens; the target encoder receives the full image under no-gradient evaluation. Target positions become predictor queries. The loss updates the predictor and online encoder, then an EMA update advances the target encoder.

4.17. What a JEPA gives a world model

The chapter began with an encoder and the question of how it can learn relations between parts of an observation. A JEPA answers by defining a context, a target and a feature space in which the target is predicted. The method moves the learning objective away from reproducing every pixel and toward matching a learned representation. That change is useful, but it does not by itself produce a physical state, an action-conditioned transition model or a complete planner.

4.17.1. What is the target?

I-JEPA selects spatial target blocks from an image. V-JEPA selects spatiotemporal target blocks from a video. VL-JEPA shows that the target representation can belong to another modality. These methods differ in their conditional prediction problems because they differ in what information is visible, what is withheld and what the target encoder represents.

The mask and query are part of the objective. A mask that makes the target easy may encourage shortcuts; a mask that removes task-relevant evidence may make the target unidentifiable. A loss is meaningful only after the sampling distribution and target representation have been specified.

4.17.2. What prevents a trivial solution?

Agreement alone admits a constant representation. An EMA target, stop-gradient or predictor changes the optimization dynamics but does not automatically prove non-collapse. Variance and covariance penalties constrain second-order geometry. SIGReg constrains projected distributions through characteristic functions. VISReg separates centre, scale and normalized shape.

These constraints address different mathematical properties. None establishes that the representation preserves the information required by a particular downstream control task. That stronger requirement is predictive sufficiency relative to a target, action set and time horizon.

4.17.3. Why does embedding geometry matter?

A covariance spectrum with weak directions can make linear readouts poorly conditioned. Local nonlinear estimators depend on where samples lie within the embedding space, leading to the Fisher-information functional under the assumptions studied by LeJEPA. Gaussian and isotropic optimality follow from distinct inequalities, and only for the risk functionals and constraints analysed.

SIGReg and VISReg make these distributional targets trainable through sampled one-dimensional comparisons. Their objectives improve control over embedding geometry, but a finite sketch does not certify exact Gaussianity and a Gaussian embedding is not automatically sufficient for physical prediction.

4.17.4. What changes when the system acts?

A passive video predictor estimates future features under the trajectory distribution in its data. An action-conditioned predictor must receive the candidate action and be trained on observations aligned with that action. Identifying its response requires state sufficiency, action support and assumptions that make observed transitions informative about interventions.

Planning then introduces another objective. The transition model predicts consequences; the cost evaluates desirability; the planner searches action sequences; feedback corrects the state estimate after execution. More search can exploit model error, and latent goal similarity is not a safety guarantee.

LeCun's proposed agent architecture adds Perception, World Model, Cost, Actor, Short-Term Memory and Configurator as distinct responsibilities. Mode 1 and Mode 2 separate fast reactive behaviour from deliberate planning. These are architectural proposals and research questions, not evidence that a complete general agent has been built.

4.17.5. Questions to carry forward

The most useful way to read a new JEPA paper is to reconstruct its information flow. What does the context encoder observe? What does the target encoder observe? How is the target selected? What does the predictor receive? What loss is applied, and which parameters receive its gradient? What distribution of contexts and targets is sampled? Finally, what downstream claim is supported by the evaluation?

Those questions keep four levels separate: representation learning, target prediction, controlled dynamics and planning. A result at one level may support the next, but it does not automatically establish competence at the others.

4.17.6. Exercises

1. **Conditional prediction.** For a squared-error feature predictor, derive the conditional-mean solution. Construct a two-mode target distribution for which the conditional mean is not a likely target.
2. **Mask design.** Describe two masks for the same video. Explain what information each leaves available and what shortcut each may permit.
3. **Collapse.** Show that a constant representation minimizes a pure view-agreement loss. Give an example of a rank-deficient representation with nonzero variance in every coordinate.
4. **Predictive sufficiency.** Construct two histories that map to the same embedding but produce different future distributions under the same action. Explain why the representation is insufficient.

5. **Ridge bias.** Derive the coefficient shrinkage factor in the eigenbasis of the covariance matrix. Which task direction has the greatest shrinkage?
6. **OLS variance.** Under the homoskedastic linear model, derive the trace of the OLS coefficient covariance. Use Jensen's inequality to show why isotropy minimizes it at fixed trace.
7. **Local regression.** Starting from the kernel-regression expectation, identify where the density gradient enters the second-order expansion. State the smoothness assumptions needed for the expansion.
8. **SIGReg.** Explain why the Cramér–Wold theorem requires all directions, while a minibatch objective samples only finitely many. Distinguish the maximum directional test from the mean directional training loss.
9. **VISReg.** Explain why the scale loss and shape loss act on different quantities. What changes when the standard-deviation denominator is stop-gradient?
10. **Intervention.** Give an example in which $p(Y \mid X, A = a)$ differs from $p(Y \mid X, \mathrm{do}(A = a))$. State what additional assumptions would allow observational trajectories to identify the intervention distribution.
11. **Model exploitation.** Construct a situation in which an optimizer finds an action sequence with low predicted cost because the transition model is wrong. Explain why increasing the number of search samples can make the selected action worse in the real system.
12. **Agent architecture.** Separate the responsibilities of World Model, Cost and Actor. Explain what Mode 2 can provide to Mode 1 through distillation, and why this does not transfer a guarantee of correctness.

Figure 4.29. Representation, prediction, dynamics and planning are separate interfaces.

Description: The encoder determines what information is represented; the JEPA objective specifies a target to predict; the controlled transition model estimates consequences under actions; and the planner selects actions using a separate cost. No single feature loss supplies all four responsibilities.

4.18. References

This file identifies the exact versions supplied for Chapter 4 and the primary sources used in the text. Paper-specific claims should be cited to the version that supports them. Where a method is a stage within a larger paper rather than a separate publication, it is cited to that paper rather than treated as an independent work.

Primary papers supplied for the chapter

4.19. I-JEPA

Assran, M., Duval, Q., Misra, I., Bojanowski, P., Vincent, P., Rabbat, M., LeCun, Y., and Ballas, N. (2023). ``Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture." *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. arXiv:2301.08243, version 3.

Supplied file: `rough-draft/chapter4/papers_pdfs/2301.08243v3.pdf`.

Primary source: https://arxiv.org/abs/2301.08243

Official implementation: https://github.com/facebookresearch/ijepa

Use in this chapter: spatial context-to-target prediction, multiple target blocks, feature-space prediction, target-encoder pathway and the limits of image-only representation learning.

4.20. Original V-JEPA

Bardes, A. et al. (2024). ``Revisiting Feature Prediction for Learning Visual Representations from Video." arXiv:2404.08471, version 1.

Supplied file: `rough-draft/chapter4/papers_pdfs/2404.08471v1.pdf`.

Primary source: https://arxiv.org/abs/2404.08471

Official project and implementation sources are maintained by Meta FAIR through the V-JEPA research releases.

Use in this chapter: spatiotemporal feature masking and prediction, target/context information flow, video representation learning, and the distinction between masked video prediction and action-conditioned dynamics.

4.21. V-JEPA 2 and V-JEPA 2-AC

Assran, M. et al. (2025). ``V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning." arXiv:2506.09985, version 1.

Supplied file: `rough-draft/chapter4/papers_pdfs/2506.09985v1.pdf`.

Primary source: https://arxiv.org/abs/2506.09985

Official implementation: https://github.com/facebookresearch/vjepa2

Use in this chapter: passive video pretraining, the separate action-conditioned post-training stage, robot trajectory data, proprioceptive inputs, image-goal planning and the scope of the reported physical deployments. V-JEPA 2-AC is treated as a model/stage within this paper, not as an

unrelated publication.

4.22. LeJEPA and SIGReg

Balestriero, R. and LeCun, Y. (2025). ``LeJEPA: Provable and Scalable Self-Supervised Learning Without the Heuristics." arXiv:2511.08544, version 3, 14 November 2025.

Supplied file: `rough-draft/chapter4/papers_pdfs/2511.08544v3 (1).pdf`.

Primary source: https://arxiv.org/abs/2511.08544

Official project/code pointer: https://github.com/rbalestr-lab/lejepa

Use in this chapter: the conditions under which a predictor is justified, multi-view agreement, linear and nonlinear probe risk, the Fisher-information functional, Gaussian optimality under stated assumptions, Cramér–Wold projections, the Epps–Pulley statistic and SIGReg.

4.23. VL-JEPA

Chen, D., Shukor, M., Moutakanni, T., Chung, W., Yu, L., Kasarla, T., Bolourchi, A., LeCun, Y., and Fung, P. (2025/2026). ``VL-JEPA: Joint Embedding Predictive Architecture for Vision-Language." arXiv:2512.10942, version 2.

Supplied file: `rough-draft/chapter4/papers_pdfs/2512.10942v2.pdf`.

Primary source: https://arxiv.org/abs/2512.10942

Use in this chapter: one cross-modal example showing that the target representation need not be visual. VL-JEPA predicts a text embedding from visual context and can decode language when a text output is required. It is not treated as an action-conditioned physical dynamics model.

4.24. V-JEPA 2.1

Mur-Labadia, L. et al. (2026). ``V-JEPA 2.1: Unlocking Dense Features in Video Self-Supervised Learning." arXiv:2603.14482, version 3.

Supplied file: `rough-draft/chapter4/papers_pdfs/2603.14482v3_compressed.pdf`.

Primary source: https://arxiv.org/abs/2603.14482

Source license: arXiv identifies version 3 as CC BY-NC-ND 4.0. Treat the paper’s figures as non-commercial, no-derivatives material; independently drawn explanatory figures are preferable.

Official implementation: https://github.com/facebookresearch/vjepa2

Use in this chapter: dense visible- and masked-token supervision, deep self-supervision and the distinction between dense representation learning and action-conditioned robot prediction. Any planning comparison must retain the predictor and planner settings from the paper.

4.25. VISReg

Wu, H., Balestriero, R., and Levine, M. (2026). ``Variance-Invariance-Sketching Regularization for JEPA Training." arXiv:2606.02572, version 1.

Supplied file: `rough-draft/chapter4/papers_pdfs/2606.02572v1 (1).pdf`.

Primary source: https://arxiv.org/abs/2606.02572

Source license: arXiv identifies the source with CC BY-NC 4.0 terms. Check the precise reuse conditions before adapting or reproducing any paper material; use original explanatory artwork.

Project/source pointer: https://haiyuwu.github.io/visreg/

Use in this chapter: explicit separation of centre, scale and shape; stop-gradient in shape normalization; sliced-Wasserstein projections; scale-gradient analysis; compute dependence on batch size, embedding dimension and number of slices; and the paper's bounded experimental claims.

4.26. Autonomous machine intelligence

LeCun, Y. (2022). *A Path Towards Autonomous Machine Intelligence*. Version 0.9.2, 27 June 2022. OpenReview: BZ5a1r-kVsf.

Supplied file: `rough-draft/chapter4/papers_pdfs/10356_a_path_towards_autonomous_mach.pdf`.

Primary source: https://openreview.net/forum?id=BZ5a1r-kVsf

Use in this chapter: energy-based compatibility, latent variables for alternative futures, the six-module agent proposal, Mode 1 and Mode 2, Mode-2-to-Mode-1 distillation, associative short-term memory, hierarchical prediction and the Configurator. The document is identified as a position paper and research proposal.

Additional background citations used in Chapter 4

These papers are cited briefly for mechanisms already introduced in Chapter 3. The Chapter 4 discussion is limited to the role each mechanism plays in the collapse argument.

- Chen, T., Kornblith, S., Norouzi, M., and Hinton, G. (2020). ``A Simple Framework for

Contrastive Learning of Visual Representations." *Proceedings of the 37th International Conference on Machine Learning (ICML)*. arXiv:2002.05709.

- Caron, M., Misra, I., Mairal, J., Goyal, P., Bojanowski, P., and Joulin, A. (2020). ``Unsupervised Learning of Visual Features by Contrasting Cluster Assignments." *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2006.09882.
- Grill, J.-B. et al. (2020). ``Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning." *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2006.07733.
- Chen, X. and He, K. (2021). ``Exploring Simple Siamese Representation Learning." *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. arXiv:2011.10566.
- Zbontar, J. et al. (2021). ``Barlow Twins: Self-Supervised Learning via Redundancy Reduction." *Proceedings of the 38th International Conference on Machine Learning (ICML)*. arXiv:2103.03230.
- Bardes, A., Ponce, J., and LeCun, Y. (2022). ``VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning." *International Conference on Learning Representations (ICLR)*. arXiv:2105.04906.